The Optimization of Pauli Word Grouping for Measuring Hamiltonian Expectation Values using QAOA

CS 4993 Honours Project

Benjamin Talbot

Department of Computer Science
University of New Brunswick, Saint John NB
btalbot@unb.ca

April 2026

Supervisors:
Dr. Stijn De Baerdemacker
Dr. Christopher Baker

Readers:
Dr. Georgiy Krylov
Dr. Rebecca McKay

Abstract. Quantum computing is a new form of computation that promises to offer improvements in processing and problem-solving currently unattainable by classical computers. Many quantum algorithms have been devised over the past few decades which exemplify this ability. The Variational Quantum Eigensolver (VQE) is one such algorithm. It is used in quantum chemistry, simulations, and optimization problems to find the ground state, or lowest energy state, of a given Hamiltonian. A Hamiltonian is an operator that describes the energy of a system, and part of VQE involves measuring the expectation value of the system's Hamiltonian to determine its ground state. This measurement process can be quite costly, with the number of measurements required using a naive approach scaling as $O(N^4)$, where N is the number of qubits (quantum bits) used to represent the system. Many methods have been devised to reduce this count. The Pauli term grouping approach is one in which the Hamiltonian terms are grouped based on commutativity and measured simultaneously. This Honours project explores this approach through the use of quadratic unconstrained binary optimization (QUBO) and the application of the Quantum Approximate Optimization Algorithm (QAOA) to a graph colouring representation of the problem. These methods did not yield valid groupings of Pauli terms, even for the smallest system considered, H_2. Using QAOA to solve the Pauli grouping problem is not a practical approach in the near term due to the limitations of current classical and quantum technology.

Keywords: VQE · QUBO · QAOA · Optimization · Quantum Computing · Variational methods · NISQ devices

Table of Contents

1 Acknowledgements . . . 3
2 Introduction . . . 3
  2.1 Motivation and Problem Statement . . . 4
  2.2 Scope and Limitations . . . 13
  2.3 Literature Review . . . 13
3 Methodology . . . 15
4 Results . . . 22
5 Discussion . . . 25
6 Conclusion . . . 28
  6.1 Summary . . . 28
  6.2 Future Work . . . 28
7 Note on LLM Usage . . . 30
8 Appendix . . . 30
9 Glossary . . . 35

List of Tables

1 Overview of VQE measurement-reduction approaches... . . . 16
2 Molecular Hamiltonian data used in this project. N is the number of... . . . 17
3 Parameters of the Hamiltonians and graph colouring problems for the... . . . 22
4 Results from classical graph colouring algorithms applied to Pauli... . . . 22
5 H_2 simulation results. . . . 23
6 QPU backends and specifications. . . . 25
7 H_2 quantum hardware results. . . . 25
8 Resource requirements for the larger molecules. The number of qubits... . . . 25

List of Figures

1 Measuring in the X basis via basis rotation. . . . 6
2 Entangled measurement circuit for ZZ and XX. . . . 7
3 Graph representations of the QWC problem. . . . 8
4 The adjacency matrix of the complement graph shown in Figure 3b. . . . 9
5 Graph representations of the QWC problem. . . . 11
6 A diagram of the QAOA algorithm. The parameterized quantum circuit is... . . . 12
7 Complement QWC graph for H_2. . . . 17
8 Research methodology flowchart. . . . 18
9 Quantum algorithm execution flowchart. . . . 21
10 Invalid vs. valid H_2 colourings. . . . 24

List of Listings

1 QUBO QuadraticProgram code for one-hot encoding. 31
2 QAOA runner code for one-hot encoding. 33
3 Cost Hamiltonian code for space-efficient embedding. 33
4 Projector-term construction code. 34
5 QAOA runner code for space-efficient embedding. 35

1 Acknowledgements

First and foremost, I would like to thank my supervisor, Dr. Stijn De Baerdemacker, for his guidance and support throughout this project. His expertise and insights were invaluable both in initiating this project and in helping it progress. It was a pleasure to work with him and to learn from him. I would also like to thank the rest of my supervisory committee, Dr. Christopher Baker, Dr. Georgiy Krylov, and Dr. Rebecca McKay, for their valuable feedback and support throughout the project. Furthermore, I would like to thank Amir Ayati, one of the members of QuNB, the quantum chemistry lab which Dr. De Baerdemacker leads, for his help with the code for generating the molecule Hamiltonians. Finally, I would like to thank my family and friends for their invaluable encouragement, support, and feedback throughout this project and my studies.

2 Introduction

Quantum computing is an emerging field that has the potential to revolutionize computing. It promises to offer computational power that is unachievable by classical computers, allowing for problems and simulations that are currently intractable to be solved. Quantum computing leverages the principles of quantum mechanics to perform computations, allowing the manipulation of information in ways which are not possible in its classical counterpart[32].

This emerging technology uses quantum bits, or qubits, which are the quantum analog of classical bits. Unlike classical bits, which can only be in a state of 0 or 1, qubits can exist in a superposition of states, meaning they can be in a combination of both 0 and 1[25]. One way to think of superposition is to imagine a coin that is spinning in the air. While it is spinning, it is not just heads or tails, but rather a combination of both; either outcome could occur. The coin only "collapses" to either of the states heads or tails when it lands. Despite computing with qubits in superposition, the result of a quantum computation is always a classical bitstring, which is obtained by measuring the qubits at the end of the computation. Similarly to the coin toss analogy, when the computation is being performed, the qubits are in a superposition of states, just like the coin in the air, but when the computation is complete and the qubits are measured, they collapse to a specific state, just like when the state of the coin is observed after it lands on either heads or tails.

Another important property of quantum computing is entanglement, which is a phenomenon where the state of one qubit can be correlated with the state of another qubit, even if they are physically separated. A consequence of this is that if two qubits are entangled, measuring one qubit will instantaneously affect the state of the other qubit[25]. For example, two qubits can be entangled in such a way that they are always in opposite states. In this case, if one qubit is measured and found to be in the state 0, the other qubit will immediately assume the state 1, regardless of the physical distance between them. Similarly, if the first qubit is measured and is in the state 1, the other will be in the state 0. This phenomenon has been experimentally verified and is a fundamental aspect of quantum mechanics[6].

Two major forms of quantum computing are gate-based architectures and quantum annealing. Gate-based architectures are called universal quantum computers because they can perform arbitrary quantum computations[32], while quantum annealing is a specialized form of quantum computing that is designed to solve optimization problems following the adiabatic theorem[4], which is described further below. In gate-based quantum computing, quantum algorithms are implemented using quantum gates in quantum circuits, which are the quantum analog of classical logic gates and circuits. These gates manipulate the state of qubits to perform computations. The resulting state is then measured to obtain a result.

Quantum algorithms are usually designed to take advantage of the abovementioned properties of quantum computing to solve problems more efficiently than classical algorithms. For example, Shor's algorithm is a quantum algorithm that can factor large integers in polynomial time, faster than the best-known classical algorithms[30]. One quantum algorithm which has proved useful in the field of quantum chemistry is the Variational Quantum Eigensolver (VQE). VQE is a hybrid quantum-classical algorithm that is used to find the ground state energy, or lowest energy state, of a quantum system, which is a fundamental problem in quantum chemistry. In essence, VQE works by preparing a parameterized quantum state (a quantum state that depends on a set of parameters) on a quantum computer and then measuring the expectation value of the Hamiltonian of the system[27]. The Hamiltonian of a quantum system is a mathematical operator that represents the total energy of the system, and measuring the expectation value of a given quantum state relative to a Hamiltonian governing the system's energy provides the (average) energy of that state. Whereas the quantum part of the algorithm sets up the state for measurement, the classical portion consists of an optimizer that optimizes the parameters of the parameterized quantum state to minimize the expectation value of the Hamiltonian, which corresponds to finding the ground state. The parameters are discussed further in the context of the Quantum Approximate Optimization Algorithm further below. Furthermore, VQE is also applicable to other types of problems, such as optimization problems, since many problems can be mapped to finding the ground state of a quantum system. How to perform such a problem mapping is briefly discussed in the context of QAOA below. The key point is that VQE is a powerful tool for solving various problems, namely optimization problems[13].

VQE is also a promising algorithm for noisy intermediate-scale quantum (NISQ) computers, which are currently limited in the number of qubits and the amount of noise they can tolerate. Performing quantum computation is currently a very delicate process. Qubits are very susceptible to noise and errors, which can lead to incorrect results. Variational algorithms have been shown to be more resilient to noise than other quantum algorithms, making VQE a promising candidate for NISQ devices[33].

2.1 Motivation and Problem Statement

Despite its usefulness, VQE has some limitations. One of the main challenges is that it requires numerous measurements to estimate the expectation value of the Hamiltonian, which can be time-consuming and resource-intensive. Given the state of NISQ devices and the fact that the output of a computation is generally probabilistic, typically quantum computations are run with many shots, meaning that the same computation is repeated many times to get a distribution of results and more accurate estimates[33]. Thus, for each computation performed, the quantum program must be initialized, the quantum circuit that implements the program and prepares the parameterized quantum state must be executed on a quantum computer, and the results must be measured and recorded. If a program is run with 1000 shots, this process is repeated 1000 times.

Another challenge with VQE, which is more significant, is that the Hamiltonian of a quantum system cannot be completely measured in a single quantum computation. Current gate-based quantum computers can only measure Hamiltonians in a certain way, and this requires the Hamiltonian to be decomposed into a different mathematical formulation (there are different types of quantum computers, gate-based qubit architectures being one of them). More specifically, taking the electronic structure problem as an example, measuring the electronic energy requires the evaluation of the following expression:

$$E_e(R) = min_{|\psi(R)\rangle} \langle \psi(R)|\hat{H}_e(R)|\psi(R)\rangle,$$

where E_e is the electronic energy, R is the nuclear configuration, $|\psi(R)\rangle$ is the electronic wavefunction, and $\hat{H}_e$ is the electronic Hamiltonian. However, this is not the exact representation used on a quantum computer. Instead of $\hat{H}_e(R)$, a quantum computer uses the qubit counterpart which we will denote as $\hat{H}$. $\hat{H}$ is obtained from the second quantized version of $\hat{H}_e(R)$. To measure the energy $E_e(R)$ of a state, a quantum computer emulates $|\psi(R)\rangle$ with the quantum state of a qubit system $|\psi\rangle$ and measures the expectation value of $\hat{H}$ with respect to $|\psi\rangle$, which is given by $\langle\psi|\hat{H}|\psi\rangle$. This is done using projective measurements, as discussed further in this section. At the base of this formulation are Pauli operators. These are operators that allow the representation and measurement of the Hamiltonian in a gate-based architecture. There are three such operators, along with the identity, which can appear in the Hamiltonian's Pauli representation. These are

$$\hat{\sigma}_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \hat{\sigma}_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \hat{\sigma}_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \text{and } \hat{\sigma}_e = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

When written as a string, these typically are represented by the characters X, Y, Z, and I, respectively. Products of these operators form Pauli words or strings: $\hat{P} = \bigotimes_{i=1}^{N} \hat{\sigma}_i$, where $\bigotimes$ is the tensor product of multiple terms (similar to Π notation) and N is the number of qubits used to represent the quantum system of interest; thus, one operator is applied to each qubit. The full Hamiltonian is then represented as a sum of these Pauli words, each with a coefficient: $\hat{H} = \sum_i c_i \hat{P}_i$. An important consideration is that the number of Pauli words needed to represent $\hat{H}$ scales as $O(N^4)$[35].

Due to the projective nature of current measurement methods in quantum computing, we can only measure one single-qubit operator $\hat{\sigma}_i$ per qubit at a time. If we measure a qubit in the Z basis, we cannot simultaneously measure it in the X or Y basis because the eigenstates of the Z operator are not eigenstates of the X or Y operators. The quantum state collapses to an eigenstate of the Z operator upon measurement, losing any superposition information that we could have investigated differently in another basis. Thus, with only single-qubit measurements, we can only measure one Pauli word at a time[31]. With N^4 Pauli words and each requiring many shots and resources to measure, the cost of measuring the Hamiltonian $\hat{H}$ can grow very quickly with the size of the problem being solved, sometimes becoming unmanageable[33]. Fortunately, it turns out that we can reduce the number of times we need to measure these Pauli strings to reconstruct the Hamiltonian. To understand how, we need to review measurement on a quantum computer in more detail.

A quantum state is a linear combination of basis states with the restriction that the squares of the coefficients, which are called probability amplitudes, sum to one (or rather the squares of the absolute values of the coefficients, since these can be complex). An arbitrary single-qubit quantum state is given as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ such that $|\alpha|^2 + |\beta|^2 = 1$ and where

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ and } |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

which are called computational basis states. A multi-qubit system is given as a linear combination of the tensor products of the single-qubit basis states: $|\psi\rangle = \sum_i c_i |b_i\rangle$ such that $\sum_i |c_i|^2 = 1$, where $|b_i\rangle$ are the basis states. For example, a two-qubit system is given as $|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle$ such that $|\alpha_{00}|^2 + |\alpha_{01}|^2 + |\alpha_{10}|^2 + |\alpha_{11}|^2 = 1$ and where $|00\rangle = |0\rangle \otimes |0\rangle$, $|01\rangle = |0\rangle \otimes |1\rangle$, $|10\rangle = |1\rangle \otimes |0\rangle$, and $|11\rangle = |1\rangle \otimes |1\rangle$. The computational basis is the basis that is chosen to represent the states $|00..00\rangle$, $|00..01\rangle$, $|00...10\rangle$, etc., which represent the possible bitstring outcomes and is typically assigned to the Z basis. Basis states are elements of a basis, though they are depicted differently than in linear algebra, typically with bra-ket (Dirac) notation rather than vector notation, as shown above. Gate-based quantum computers use projective measurements, which means that when a quantum state is measured, it collapses into one of the basis states of the system. More specifically, projective measurements are described by a set of projection operators $\{P_i\}$, where each P_i corresponds to a measurement outcome. When a quantum state $|\psi\rangle$ is measured, the probability of obtaining the outcome corresponding to P_i is given by $p_i = \langle\psi|P_i|\psi\rangle$ (which denotes the inner product between $|\psi\rangle$ and $P_i|\psi\rangle$), and the post-measurement state collapses to $\frac{P_i|\psi\rangle}{\sqrt{p_i}}$. The possible measurement outcomes are the eigenvalues of the operator being measured, and the system collapses to the corresponding operator's eigenstate[25]. One unique aspect of quantum computing is that there are different ways in which the quantum state resulting from a computation can be measured. Measuring in different ways is referred to as measuring in a different basis, since a different basis is chosen as the possible outcomes of the measurement. As a brief example, the state

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

would return the state $|0\rangle$ with probability $\frac{1}{2}$ and the state $|1\rangle$ with probability $\frac{1}{2}$ if measured in the computational basis. It can be thought of as being "halfway" between the two computational basis states. However, if we measure $|+\rangle$ in the X basis, which has eigenstates $|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$ and $|-\rangle = \frac{|0\rangle-|1\rangle}{\sqrt{2}}$, we would obtain the state $|+\rangle$ with probability 1 and the state $|-\rangle$ with probability 0. This is because $|+\rangle$ is an eigenstate of the X operator, so it will always collapse to itself when measured in the X basis[25]. Figure 1 shows a quantum circuit that effectively measures the $|+\rangle$ state in the X basis by first applying a Hadamard gate $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ and then measuring in the computational (Z) basis. The Hadamard maps X basis eigenstates to Z basis eigenstates, making measurement in the computational basis equivalent to measurement in the X basis (where the $|0\rangle$ and $|1\rangle$ outcomes in the computational basis correspond to the $|+\rangle$ and $|-\rangle$ outcomes in the X basis, respectively).

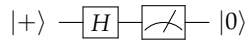


Fig. 1: Rotating the measurement basis to measure in the X basis. The meter symbol indicates measurement in the computational basis.

In the context of measuring the Hamiltonian $\hat{H}$, its decomposition is given as a sum of Pauli words $\hat{P}_i$, each of which indicates the single-qubit operators that need to be measured on each qubit. Each Pauli operator has its own eigenbasis, so to measure a Pauli word, we need to rotate the measurement basis of the qubits to the eigenbasis of their corresponding Pauli operators. For example, if we want to measure a Pauli word that contains an X operator on a particular qubit, we would need to use the above circuit to measure that qubit in the X basis. Similarly, if we want to measure a Y operator on a qubit, we would need to apply a specific unitary transformation (a combination of gates) to rotate the Y basis to the Z basis before measurement. The identity operator I does not require any rotation since it does not affect the state of the qubit[35].

As described above, a Pauli string is a tensor product of single-qubit Pauli operators, each of which has its own eigenbasis. A tensor product basis (TPB) is a basis for the entire system that is formed by taking the tensor product of the eigenbases of the individual Pauli operators. For example, if we have a two-qubit system and we want to measure the Pauli word XZ, we would need to rotate the first qubit to the X basis and the second qubit to the Z basis. The measurement would then be performed in the tensor product basis formed by these two bases. As an example of a TPB, the eigenbasis of the Pauli word ZZ would be $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, where $|0\rangle$ and $|1\rangle$ are the eigenstates of the Z operator and $|\cdot\cdot\rangle$ is used as a shorthand for the tensor product $|\cdot\rangle \otimes |\cdot\rangle$. By measuring in this TPB, we can obtain information about both the Z components of the state simultaneously (the Z on the first qubit and the Z on the second qubit). The same is true for other Pauli words, such as XX, XY, etc., each of which has its own TPB. However, we cannot measure Pauli words that do not share the same TPB simultaneously. For example, if we want to measure the Pauli words ZZ and XX, we cannot measure them together since they do not share the same TPB. Their shared eigenstates are

$$\frac{|00\rangle + |11\rangle}{\sqrt(2)}, \frac{|00\rangle - |11\rangle}{\sqrt(2)}, \frac{|01\rangle + |10\rangle}{\sqrt(2)}, \text{ and } \frac{|01\rangle - |10\rangle}{\sqrt(2)},$$

which are not eigenstates of either ZZ or XX; these are entangled eigenstates. Thus, in order to measure these together, we would need to perform entangled measurements. These are more complex and require larger circuits, with a quadratic scaling in the number of gates required. For the case above, we would need to perform a Bell measurement, which is a specific type of entangled measurement that can be used to measure the shared eigenstates of ZZ and XX. It requires a CNOT gate followed by a Hadamard gate and then measurement in the computational basis[14]. The quantum circuit in Figure 2 shows this process. The CNOT gate is represented by the following matrix:

$$= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Entangled measurements are not used in this project for reasons discussed below regarding commutativity.

The crucial point is that we can gain information about multiple Pauli words simultaneously if they share the same TPB, meaning that they can be measured together in the same measurement. The condition that allows us to do this is that each corresponding (same index) operator in the Pauli words must either be the same or be an identity. This condition will be elaborated below. The problem then becomes how to optimally group the Pauli words in the Hamiltonian such that we can measure as many of them together as possible, thus reducing the number of measurements required to obtain the expectation value of the Hamiltonian. This results in an optimization problem, which we call the Pauli grouping problem.

To help solve this optimization problem, Verteletskyi et al. use the qubit-wise commutator. This denotes whether two Pauli words $\hat{P}_i = \bigotimes_k \hat{\sigma}_k$ and $\hat{P}_j = \bigotimes_k \hat{\sigma}'_k$ are qubit-wise commuting (QWC), meaning that all one-qubit

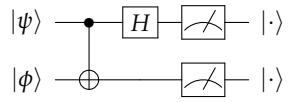


Fig. 2: An entangled measurement circuit that can be used to measure the shared eigenstates of ZZ and XX.

operators in $\hat{P}_i$ commute with their counterparts in $\hat{P}_j$ (they are either the same or an identity)[33]. The qubit-wise commutator is defined as follows:

$$[\hat{P}_i, \hat{P}_j]_{\text{qw}} = \begin{cases} 0, & \text{if } [\hat{\sigma}_k, \hat{\sigma}'_k] = 0 \; \forall \, k \\ 1, & \text{otherwise.} \end{cases}$$

To reduce the number of measurements required to obtain the expectation value of a Hamiltonian, we can group Pauli terms together such that each group contains terms that are mutually QWC. This is a necessary and sufficient condition for the Pauli words to share a TPB. If we denote $\hat{A}_i$ as a group of mutually QWC terms, then we can represent the Hamiltonian $\hat{H}$ as

$$\hat{H} = \sum_i \hat{A}_i,$$

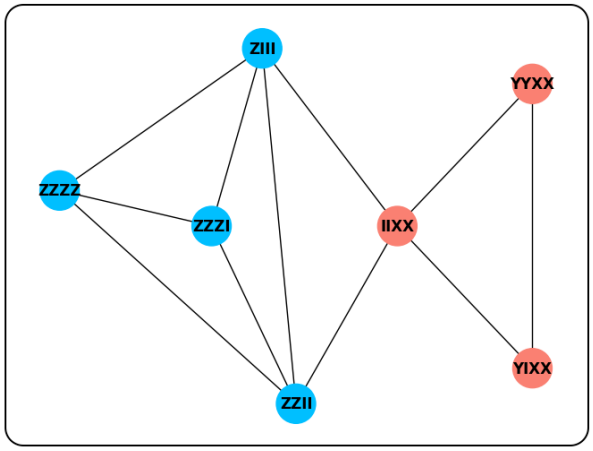
where each $\hat{A}_i = \sum_i c_i \hat{P}_i$ and $[\hat{P}_i, \hat{P}_j]_{\text{qw}} = 0$ within each $\hat{A}_i$. By representing $\hat{H}$ in this way, we can gain information about all Pauli words within each group $\hat{A}_i$ with a single measurement[35]. The goal of representing the Hamiltonian in this manner is to find a method to optimally group the terms to minimize the number of resulting groups $\hat{A}_i$, thus requiring the minimum number of measurements to obtain the expectation value of $\hat{H}$.

We can also use general commutativity to group Pauli words together as described in [14] and [33]. We do not use this approach for two reasons. First, general commutativity grouping requires entangled measurements, which require more complex circuits and more gates, as described above. Second, and more importantly, we compare our results to those obtained by Verteletskyi et al. who use qubit-wise commutativity, so we use the same approach for consistency and accurate comparison.

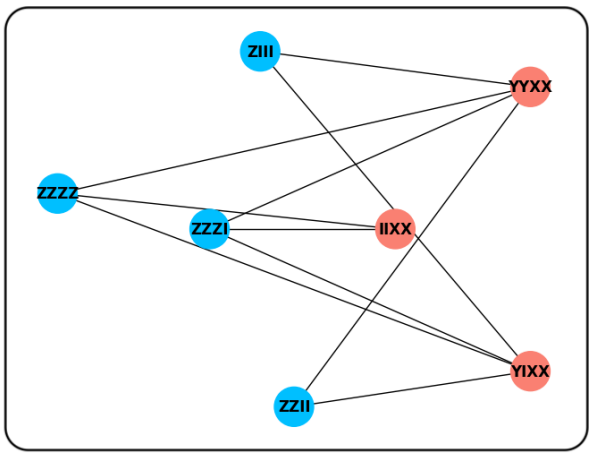
By using the QWC relation, we can model a Hamiltonian's terms as a graph in which vertices represent the terms and edges between vertices indicate that those Pauli words are QWC, meaning they can be measured together. In the graph, a group of mutually commuting vertices is depicted by a clique, a fully-connected sub-graph. The problem of optimizing the grouping of Pauli operators is thus equivalent to the minimum clique cover (MCC) problem, which involves finding the minimum number of cliques covering an entire graph[35].

The MCC problem is NP-hard in general, so no known efficient algorithm exists to solve it, where an efficient algorithm is defined as one that runs in polynomial time and space. Polynomial-time algorithms exist which use heuristics to provide approximate solutions to the problem, and this has been investigated in previous research[35]. These approaches will be discussed in more detail in the next section. The approach used in this current paper involves using a connection to the graph colouring problem as described in Verteletskyi et al[35]. The graph colouring problem involves finding the minimum number of colours required to colour the vertices of a graph such that no connected vertices share the same colour. The MCC problem can be mapped to the graph colouring problem by noticing that the minimum number of cliques in a graph G is equal to the minimum number of colours required to colour its complement graph $\bar{G}$ since each clique in G corresponds to vertices in $\bar{G}$ of the same colour. Thus, we can investigate solving the graph colouring problem to find the minimum number of groups of QWC Pauli operators by taking the complement graph of the graph formed by the Pauli terms. The graphs in Figure 3 below show an example graph of Pauli terms in which the cliques are coloured to show different groups of mutually QWC terms. They demonstrate the relationship between the MCC and graph colouring problems. Figure 3a is the normal QWC graph, whereas Figure 3b is the complement QWC graph and is the output of the graph colouring algorithm. As can be seen by this colouring, the minimum number of cliques covering the graph is 2.

Two approaches are used in this report to implement the graph colouring problem to solve the Pauli grouping problem. Both involve representing the constraints of the problem as a cost function. Such a function is evaluated on a given solution of the problem, which corresponds to a given graph colouring, and the associated cost indicates



(a) An example Pauli term graph with the minimum clique cover highlighted.



(b) The complement graph of the QWC graph with QWC groups coloured.

Fig. 3: Graph representations of the QWC problem.

whether the solution is optimal, in which case the cost would be 0, or suboptimal, which is given by a positive cost. The optimization problem becomes finding a solution with the minimum cost. One method, called the one-hot encoding approach, encodes each vertex-colour pair as a binary variable $x_{v,i}$ which indicates whether vertex v is assigned colour i. For each vertex, only one of these should be a 1 and the rest 0 to satisfy the constraint of assigning a single colour to each vertex. Thus, in a graph colouring problem with n vertices and k colours, there are nk variables (which will become qubits in the implementation used in this project). The one colour per vertex constraint is implemented via the term

$$\sum_{v=i}^{n} \left(1 - \sum_{i=1}^{k} x_{v,i}\right)^2,$$

where the inner sum is over the colours, which should only sum to one under the constraint, and the outer sum is over the vertices. The constraint of no neighbouring vertices sharing a colour is captured by the term

$$\sum_{v,w=1}^{n} \sum_{i=1}^{k} A_{vw} x_{v,i} x_{w,i},$$

where $A_{v,w}$ is the entry in the graph's adjacency matrix that depicts whether the two vertices v and w are connected by an edge. The adjacency matrix for the complement graph shown in Figure 3b is shown in Figure 4. A 1 in row

r and column c indicates that vertices r and c are connected by an edge, but since this is for the complement graph, it indicates that they are not QWC. For reference, the Pauli terms map to the indices of the rows/columns of the adjacency matrix in the order ZZZZ, ZIII, ZZZI, ZZII, IIXX, YYXX, YIXX. The purpose of including $A_{v,w}$ in the double sum expression above is to ensure that only connected vertices are penalized for sharing a colour. If $A_{v,w} = 1$, then the term $x_{v,i}x_{w,i}$ being 1 indicates that these two connected vertices share a colour, and the cost increases due to this. Inspecting the graph can help to understand this. For example, vertices 1 and 2 are not connected, so if they share a colour, the cost does not increase since $A_{1,2} = 0$. However, vertices 1 and 5 are connected, so it does matter if they share a colour and the cost increases since $A_{1,5} = 1$.

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Fig. 4: The adjacency matrix of the complement graph shown in Figure 3b.

The double sum term ensures that valid colourings result in a value of 0 by requiring the two vertices v and w to not share the same colour i if they are connected. The overall cost function. for the graph colouring problem is

$$f(x) = C \sum_{v=i}^{n} \left(1 - \sum_{i=1}^{k} x_{v,i}\right)^2 + D \sum_{v,w=1}^{n} \sum_{i=1}^{k} A_{vw} x_{v,i} x_{w,i}, \tag{1}$$

where C and D are arbitrary positive constants that can be tuned to emphasize either the single colour per vertex or the no neighbouring vertices sharing a colour constraint, respectively. These act as penalty weights, making the corresponding terms in the cost function costly when the constraints are violated[31].

The representation of the graph colouring problem as Equation 1 is a quadratic unconstrained binary optimization (QUBO) problem. QUBO problems are a type of combinatorial optimization problem (COP), which are the broader category of problems which QAOA, a quantum algorithm used in this project, is designed to solve[13]. COPs are problems in which the objective is to find an optimal solution to a problem from a set of possible solutions[21]. Typically, these types of problems are not tractable through exhaustive search, so special algorithms are used. A QUBO problem is a type of COP in which the goal is to minimize a quadratic objective (or cost) function of binary variables without any constraints (such as $Ax \leq b$)[11]. The general form of a QUBO problem is given by:

$$\text{minimize } x^T Q x + c^T x, \tag{2}$$

where $x \in \{0,1\}^n$ (i.e. x is a vector of n binary digits), Q is an $n \times n$ matrix of constants, and c is a vector of linear coefficients. Each solution vector x can be plugged into the objective function to evaluate its optimality as a solution. The lower the resulting objective function evaluation, or cost, the more optimal the solution. Though QUBO problems are unconstrained, constraints can be added to a QUBO problem by adding penalty terms, terms in the objective function which add to the cost if the constraints are not satisfied.

The second approach used to implement the graph colouring problem involves representing the problem as a Hamiltonian. This method is called the space-efficient embedding of the problem since it uses only $n\lceil \log_2(k) \rceil$ qubits. It achieves this by encoding the vertex-colour combinations as binary numbers. Each colour is assigned its own binary number, requiring $m = \lceil \log_2(k) \rceil$ bits for k colours. Each vertex is assigned the same number of bits as the colours, $b_{v,1}, b_{v,2}, \ldots, b_{v,m}$, and the bitstring formed by these bits encodes the colour assigned to vertex v. This also ensures the one colour per vertex constraint is satisfied since the bits associated to each vertex can only encode one binary number and thus one colour. To ensure that no neighbouring vertices share the same colour, additional

terms are used. As an example, in the case of 4 colours, the following term is used to ensure that vertices v and w do not share the same colour:

$$b_{v,1}b_{w,1}b_{v,2}b_{w,2} + (1-b_{v,1})(1-b_{w,1})(1-b_{v,2})(1-b_{w,2})$$
$$+(1-b_{v,1})(1-b_{w,1})b_{v,2}b_{w,2} + b_{v,1}b_{w,1}(b_{v,2}-1)(b_{w,2}-1).$$

This expression is only 0 if no neighbouring vertices share the same colour, and it is greater than 0 otherwise. In general, following the procedure in [31], the resulting Hamiltonian for the space-efficient embedding of the graph colouring problem is

$$H = \sum_{v,w=1}^{n} A_{vw} \sum_{a\in\{0,1\}^m} \prod_{\ell=1}^{m} (\mathbb{1} + (-1)^{a_\ell} Z_{v,\ell})\,(\mathbb{1} + (-1)^{a_\ell} Z_{w,\ell})\,. \tag{3}$$

Additional penalty terms can be added to penalize non-allowed bitstrings in the case that the number of colours is not a power of 2 as described in [10]. The space-efficient representation of the problem is useful since it greatly reduces the number of qubits required to represent the problem which in turn reduces the resources required to solve the problem. However, it also results in a more complex Hamiltonian and consequently a more complex quantum circuit and implementation, which also increases the depth of the quantum circuit and thus the noise and errors that can occur in the computation.

Given the higher order terms in the Hamiltonian, the space-efficient embedding of the graph colouring problem is not a QUBO problem. It has a more physical interpretation as the Hamiltonian of an Ising model[31], which is a mathematical model of ferromagnetism in statistical mechanics[22]. The reason for using Ising models is discussed alongside QAOA below. The Z_i operators in the Hamiltonian are translated into Pauli Z operators acting on the corresponding qubits when the problem is implemented on a quantum computer, and solving the problem corresponds to finding the ground state of this Hamiltonian, which can be done using QAOA as described further below.

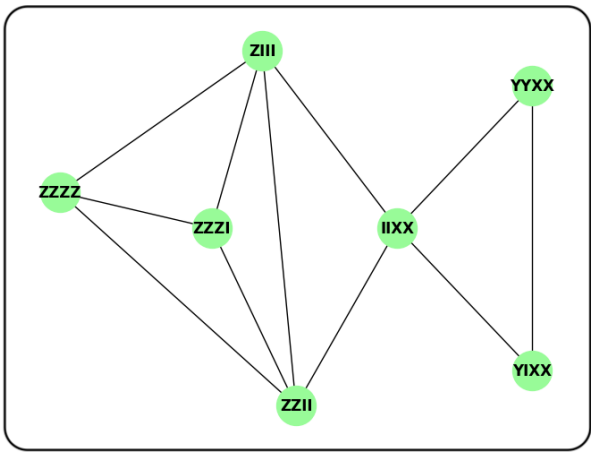
As an example of the graph colouring problem applied to the Pauli grouping problem, take the following Hamiltonian:

$$\hat{H} = \mathrm{ZZZZ} + \mathrm{ZIII} + \mathrm{ZZZI} + \mathrm{ZZII} + \mathrm{IIXX} + \mathrm{YYXX} + \mathrm{YIXX}.$$

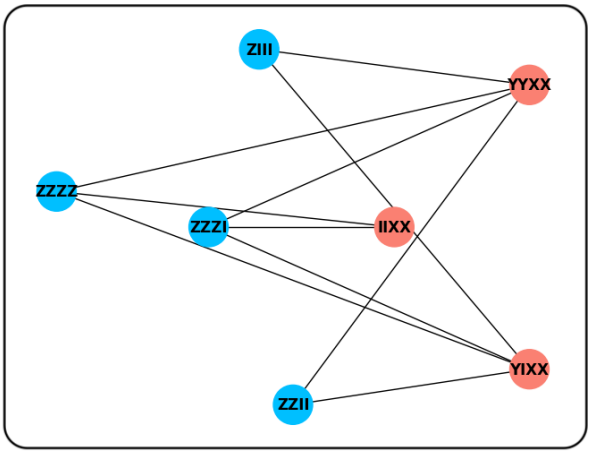
By analyzing the terms, we can determine which terms are QWC and thus can be measured together. In this case, the first four terms are mutually QWC, while the last three terms are mutually QWC. Thus, the Hamiltonian can be represented as $\hat{H} = \hat{A}_1 + \hat{A}_2$, where $\hat{A}_1 = \mathrm{ZZZZ} + \mathrm{ZIII} + \mathrm{ZZZI} + \mathrm{ZZII}$ and $\hat{A}_2 = \mathrm{IIXX} + \mathrm{YYXX} + \mathrm{YIXX}$. This means that we can measure the expectation value of $\hat{H}$ by performing two measurements, one for each group $\hat{A}_1$ and $\hat{A}_2$, instead of measuring each term separately, which would require seven measurements. The graph representation of this problem is depicted in Figure 5, in which the vertices represent the terms in the Hamiltonian and the edges represent the QWC relation between the terms. This graph is the same as the one in Figure 3, shown again for ease of reference. In Figure 5a, the normal QWC graph is depicted in which an edge connecting two terms means that those terms commute. Figure 5b depicts the complement graph, meaning edges indicate that the corresponding terms do not commute, and it is coloured to show the two groups of mutually QWC terms. Note that this second graph is the output of the graph colouring problem since the complement graph is used.

Of course, simply restating an NP-hard problem differently will not make the problem any easier to solve. The graph colouring problem is also NP-hard, so there is no advantage in using this approach over any other. However, our goal is to use the existing one-hot encoding and space-efficient formulations of the graph colouring problem and study their performance in an implementation of the Quantum Approximate Optimization Algorithm (QAOA).

The Quantum Approximate Optimization Algorithm is a hybrid quantum-classical variational algorithm that can be used to solve COPs. A QUBO problem, being a type of COP, can also be solved using this algorithm. The QUBO cost function must first be translated to an Ising Hamiltonian, also referred to as a cost Hamiltonian, one defined in terms of operators amenable to quantum computation, in order to use QAOA. This can be done using methods such as those found in [26] and [31]. The space-efficient embedding is already in the form of a Hamiltonian, so its implementation on a quantum computer is more direct. Ising Hamiltonians are used in quantum computing to represent optimization problems because they can be readily implemented on quantum hardware. An important theorem in the process used to solve problems represented this way is the adiabatic theorem, which states that a quantum system will remain in its ground state if the Hamiltonian of the system is changed slowly enough. This leads



(a) An example QWC graph.



(b) The complement graph of the QWC graph with QWC groups coloured.

Fig. 5: Graph representations of the QWC problem.

to quantum annealers which are specifically designed to implement this process for optimization problems[4]. It also leads to algorithms on gate-based quantum computers such as VQE and QAOA which are based on this theorem. Such algorithms begin with a quantum state in the ground state of a simple, easy to prepare Hamiltonian. Then, the algorithm slowly changes the Hamiltonian to the cost or Ising Hamiltonian that encodes the problem we want to solve. If the change is slow enough, the system will remain in the ground state of each successive Hamiltonian, which will eventually become the ground state of the cost Hamiltonian, thus giving us the optimal solution to the problem[4].

QAOA works by using a parameterized quantum circuit, similar to VQE, to initialize a quantum state whose expectation value is then measured. The measurements are used in the classical portion to set up a cost function that is optimized by a classical optimizer to find the best parameters for the parameterized circuit to find the optimal solution to the encoded problem. The parameterized circuit is composed of two alternating components, which are repeated a chosen number of times, called the number of repetitions p of the algorithm. The two alternating layers are a cost layer and a mixer layer. Each layer is implemented as a series of quantum gates. The cost layer comes from the cost Hamiltonian representation of the original mathematical cost function, and the mixer layer is depicted by a mixer Hamiltonian, which is defined in a way to mix the probability amplitudes of the various potential solutions. In essence, the mixer layer allows the algorithm to explore different regions of the solution space, while the cost layer guides the search towards valid answers and the optimal solution. Each layer has its own set of parameters, and these parameters are used to control the gates in the corresponding layer. In basic terms, it essentially controls how strongly

the cost layer and how strongly the mixer layer is applied to the quantum state at each step, corresponding to how long the system evolves under the layer's Hamiltonian. The parameters are optimized by the classical optimizer to find the best combination that leads to the optimal solution. The choice of mixer Hamiltonian can have a significant impact on the performance of the algorithm, and different mixers can be used depending on the problem being solved[9].

There are generally two types of mixers which can be used in QAOA: pre-defined general purpose mixers and custom mixers. Pre-defined mixers are designed to be applicable to a wide range of problems, typically implementing a straightforward mixing strategy that allows the algorithm to explore any part of the solution space without any bias. Custom mixers, on the other hand, are designed to take advantage of the specific structure of the problem being solved. They can be tailored to the problem to guide the search towards more promising regions of the solution space, potentially improving the performance of the algorithm. For example, if we know that certain solutions are more likely to be optimal based on the structure of the problem, or if we can create a constraint-preserving mixer, which moves the search only between valid solutions, we can design a custom mixer that biases the search towards those solutions. Figure 6 below depicts a diagram of the QAOA algorithm from the 2021 Qiskit Global Summer School "Introduction to the Quantum Approximate Optimization Algorithm and Applications" lecture, showing the interaction between the quantum and classical components of the algorithm[28].

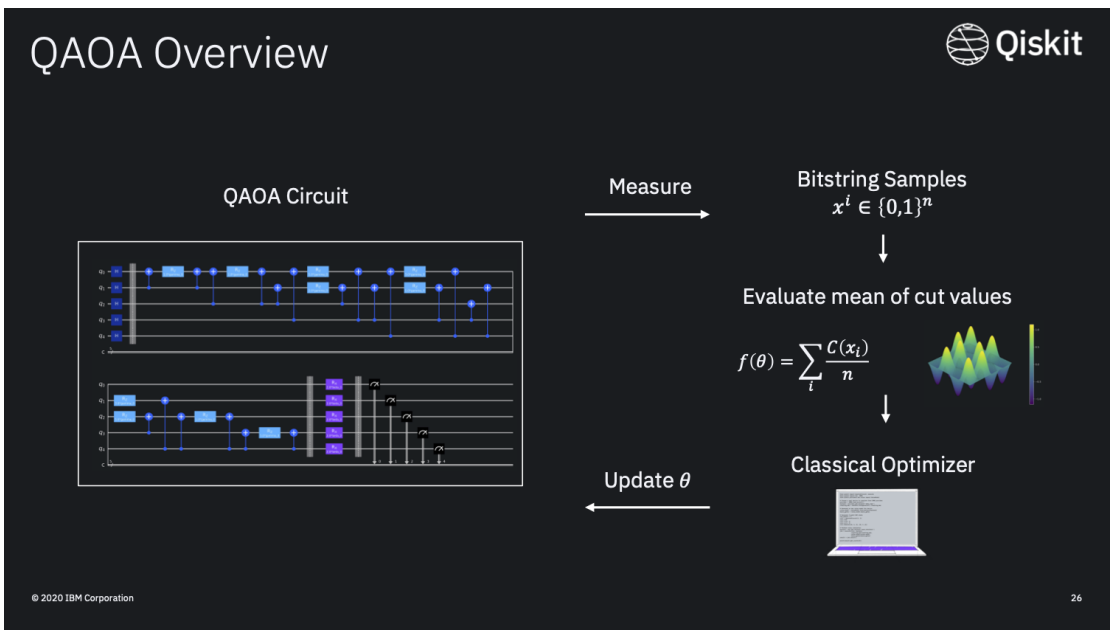


Fig. 6: A diagram of the QAOA algorithm. The parameterized quantum circuit is composed of alternating parameterized cost and mixer layers, which are optimized by a classical optimizer to find the best parameters for the circuit. The cost function is derived from the measurements performed, which are used to evaluate the expectation value of the cost Hamiltonian. The goal is to find the parameters that minimize this expectation value, which corresponds to finding the optimal solution to the original optimization problem. "Evaluate mean of cut values" is in reference to the MaxCut problem. In general, it refers to evaluating the mean of the cost function evaluations.

Another important aspect of QAOA is its implementation as a quantum circuit and the resulting depth of the circuit. The depth of a quantum circuit is the number of layers of gates that are applied to the qubits. It can have a significant impact on the performance of the algorithm, especially on NISQ devices, which are limited in their ability to tolerate noise and errors. Deeper circuits are more susceptible to noise and errors, which can lead to incorrect results. Thus, it is important to design the quantum circuit for QAOA in a way that minimizes its depth while still effectively implementing the cost and mixer layers.

The two graph colouring problem formulations have been used previously in research performed by Tabi et al. They applied the methods to colour certain random graphs by implementing the formulations as quantum annealing programs, which they ran on a D-Wave quantum annealer[31]. Our research aims to apply the QUBO and resource-efficient representations of the graph colouring problem and their quantum implementations to the Pauli grouping problem using QAOA. This will allow us to investigate the performance of these methods in solving the Pauli grouping problem and compare them to classical approaches. To the best of our knowledge, this has not been done before; quantum applications have only been explored in the context of the graph colouring problem, not the Pauli grouping problem; in particular, QAOA has not been applied to the Pauli grouping problem. Thus, this would be the first application of a quantum algorithm to optimize the grouping of Pauli terms for measurements in quantum computing, and we investigate whether QAOA is a viable method for solving this problem in the near term and whether it can provide any improvements or advantages over classical approaches.

2.2 Scope and Limitations

The scope of this project is an end-to-end implementation of the QUBO and resource-efficient formulations of the graph colouring problem and their application to the Pauli grouping problem using QAOA. The project involves the following steps, most of which are programmed in Python using the Qiskit library for quantum computing:

- Formulating the graph colouring problem as a QUBO problem and a resource-efficient formulation.
- Implementing the QUBO and resource-efficient formulations as quantum programs using QAOA.
- Testing the implementations on instances of the Pauli grouping problem applied to molecular Hamiltonians and comparing the results to classical approaches.
- Analyzing the performance of the quantum implementations and discussing their advantages and limitations.

One major limitation of this project is the short timeframe that was available to complete it. Given the infeasibility of the previous topic that was chosen for this project, which was to try to optimize unitary decomposition into quantum gates using QUBO and QAOA, the topic of optimizing Pauli grouping for Hamiltonian measurement was chosen as an alternative midway through the project. A second limitation is the limited number of qubits available on current gate-based quantum hardware. This restricted the size of the problem instances that could be run on actual quantum processing units (QPUs) to the smallest system, the hydrogen molecule. Another limitation is the computational complexity of the problem being investigated. Simulating quantum algorithms on classical computers is computationally expensive and grows quickly with the size of the problem. Thus, the instances of the Pauli grouping problem that can be investigated are limited in size. The simulation complexity also limits the performance of QAOA. QAOA is a quantum circuit that has repeated layers of parameterized gates. The number of repetitions is a parameter that can be tuned to improve the performance of the algorithm. While increasing the number of repetitions generally increases the performance, it also increases the depth of the quantum circuit, which can lead to increased noise and errors in NISQ devices as well as increased resource requirements for classical simulation.

2.3 Literature Review

When implementing VQE on a gate-based quantum computer, a molecular Hamiltonian must be mapped to qubits, requiring the Hamiltonian to be decomposed into a sum of Pauli operators. The number of terms in this decomposition typically scales as $O(N^4)$, with N qubits used in the representation. As mentioned above, the large number of terms in these qubit Hamiltonian descriptions combined with the need to measure several times makes complete measurement of the Hamiltonian extremely costly. Much research has thus been performed to discover ways to circumvent this issue. Several approaches have been pursued, including reducing the number of distinct measurements that must be performed, reducing the number of shots required to sample an expectation value with enough precision, and reducing the number of terms in the Hamiltonian and similar statistics that cause the complexity of the problem to grow quickly.

Regarding the attempts to reduce the shot count, we first start with the simple approach of distributing a limited number of shots across Pauli strings uniformly, meaning each Pauli string gets the same number of shots in its measurement. The number of shots S required to achieve a target standard error ϵ for a Hamiltonian with p terms is

$$S = p \sum_{i}^{p} \frac{c_i^2 \mathrm{Var}[\hat{P}_i]}{\epsilon^2}.$$

This is not the most optimal way of distributing shots across terms[33]. A different approach which reduces the estimation variance and thus improves the precision of the measurement is to use a weighted distribution of measurements. Instead of allocating an equal number of shots to each Pauli string, the weights of each string in the Hamiltonian's construction are used to determine how many shots should be allocated to the string. Recall that a Hamiltonian is decomposed into a sum of Pauli strings as $\hat{H} = \sum_i c_i \hat{P}_i$.

In the equation above, we can see that the contribution to the variance of the terms with larger coefficients $|c_i|$ is larger than the terms with smaller coefficients. Thus, as long as $\mathrm{Var}[\hat{P}_i]$ is similar for all i, we can redistribute terms according to these coefficients, removing shots from the terms with smaller $|c_i|$ and adding them to the terms with larger $|c_i|$, to reduce the total variance[36].

A similar method involves on-the-fly sampling of the Hamiltonian using part of the shot budget to estimate the standard deviations and variances associated with the measurement of each group of Pauli strings. These in turn can be used to allocate the remaining shots in a way that minimizes the total variance of the measurement[38].

Another method devised in McClean et al.'s work truncates the Hamiltonian by removing the Pauli terms that have contributions significantly below a chosen error tolerance threshold ϵ_t. By ordering the Pauli terms by their coefficients c_i, we can remove the smallest k terms such that $e_k < C\epsilon_t$, where $e_k = \sum_i^k |c_i|$ and $C \in [0, 1[$ is a chosen constant[24].

In addition to these term reduction or shot allocation approaches, inference methods have been developed to try to infer more information about a quantum system from fewer shots. This concept is known as shadow tomography[3]. Classical shadow tomography applies randomly-picked unitaries (unitary matrix transformations) and uses classical post-processing to infer properties of the quantum state without performing full tomography (measuring enough times to completely reconstruct the unknown state). This can help avoid the exponential scaling in some problem instances. Classical shadows can be applied to the energy estimation part of VQE. Another shadow tomography method is neural network tomography, which uses machine learning on a series of shot outputs to decrease the variance of the expectation value. This also helps reduce the number of shots required to achieve a given precision level on the measurement of a Hamiltonian[33].

Another approach used to reduce the number of terms in the Hamiltonian involves partitioning it into unitary operators. By connecting anti-commuting terms of the Pauli decomposition of a Hamiltonian, we can re-write the latter as a weighted sum of a near minimum number of unitary operators U_i as $\hat{H} = \sum_i w_i \hat{U}_i$. Such a grouping of Pauli terms into unitary operators has been shown to linearly reduce the number of terms in the Hamiltonian, allowing fewer measurements to be performed[19,33].

Another group of partitioning methods is based upon commutativity of Pauli strings to simultaneously diagonalize groups of Pauli strings through unitary rotations of the measurement basis. The goal of these strategies is to perform joint measurements on groups of commuting operators, giving the ability to simultaneously assess multiple Pauli operators in a single measurement, effectively obtaining information about several terms simultaneously. One such approach uses qubit-wise commutativity (QWC), also called tensor product basis (TPB) groups (this is the approach discussed in the previous section). As a reminder, two Pauli strings are QWC if the corresponding operators at each index commute, meaning they are either the same operator or one is the identity. As an example, the set of terms XI, IZ, and XZ are QWC. Grouping terms by QWC reduces the number of measurements required to around one third of what would otherwise be necessary[35]. In addition, the basis rotation needed to perform the joint measurements can be performed by a single layer of simple single-qubit gates appended to the circuit, before the measurement process. General commutativity (GC) is another method that uses the same idea as QWC, but using a different commutativity criteria to group Pauli terms. Instead of requiring the Pauli terms in a group to be QWC, it requires them to generally commute[14]. This differs from QWC in the fact that it only requires the number of terms that anti-commute in the group to be even, not 0. For example, the set of terms XYZ, XZY, and ZIX generally commute, but they are not QWC. This can further reduce the scale of separate terms to be measured from $O(N^4)$ to $O(N^3)$ in some cases[12]. It does, however, take a much more complicated unitary rotation to simultaneously diagonalize all the elements of a given group, one which requires non-local, entangling operations. These rotation circuits can require $O(N^2)$ gates, which is usually small but can sometimes be large in some VQE implementations[33].

Two alternative methods which are similar in nature rewrite a Hamiltonian in a basis that makes its terms simple, typically diagonal in the computational basis, so that many terms can be estimated together from a single measurement. These techniques are called decomposed interactions methods. The first method, basis rotation grouping, involves using low-rank factorization to rewrite the Hamiltonian as rank-1 factors. Unitary orbital rotations are constructed and applied to make the factors diagonal in the computational basis, meaning the expectation values of the

operators composing a factor can all be obtained through a single measurement process. Basis rotation grouping reduces the number of joint terms to measure to a linear number with system size[15]. The second approach uses Cartan subalgebras to group operators together and diagonalize them[37], resulting in a potential of achieving half the number of partitions found using basis rotation grouping[15].

Another collection of approaches is based on using heuristics on graphs to group Pauli terms. Choosing an above-mentioned rule (e.g. QWC or GC), a straightforward mapping to graphs in which vertices correspond to the Pauli terms of the Hamiltonian can be performed by assigning edges between vertices that satisfy the property. Solving the problem of grouping the Pauli terms is equivalent to the minimum clique cover problem for the corresponding graph or the graph colouring problem for the complement graph. Conventional grouping heuristics that have been used in the past specifically in the context of VQE include several graph colouring or minimum clique cover problem algorithms. Greedy colouring, largest degree first, smallest degree first, DSatur, Dutton and Brigham, COSINE, Ramsey, recursive largest first[35], and independent-operator sorting[34] are many examples of some algorithms that have been applied in this context.

A separate method directly optimizes the number of measurements required rather than the number of terms to measure. Grouping Pauli strings can result in measurement co-variances, and this increases the number of measurements needed to achieve a given precision. Sorted insertion is a technique that allocates Pauli strings to commuting groups in descending order of their absolute weights in the Hamiltonian in an attempt to reduce the introduction of measurement co-variances[7].

In summary, while VQE is a promising algorithm for NISQ devices, it comes with the caveat that measuring the expectation value of a Hamiltonian can be quite expensive. While many approaches to reduce the measurement cost focus on reducing the number of terms in the Pauli operator decomposition of the Hamiltonian, recent studies have shown that this is not necessarily the only method to do so, and some other methods might provide better results. The same is true for unitary grouping and basis rotation methods. These techniques do offer improvements over naive measurement strategies, but they suffer from co-variances which occur due to the joint measurement of terms. This increases sampling noise and consequently the number of measurements necessary to achieve a given level of precision. Additionally, the quantum circuits required in some of these methods add further noise to those programs which run on NISQ devices. Furthermore, most grouping heuristics can scale quite badly for large systems, reaching asymptotic behaviour on the order of $O(N^8)$. Inference methods could perform better but come with their own drawbacks, namely the need to train machine learning models. One final note is that sorted insertion using general commutativity typically achieves the best number of measurements to precision level ratio, while decomposed interactions methods achieve similar results with a polynomial degree less depth in the circuit implementation. As mentioned above, qubit-wise commutativity was used in this project to provide a more accurate comparison with the results obtained in Verteletskyi et al.'s research since they use this commutativity[35]. A summary of the approaches discussed in this literature review is provided in Table 1 below, which includes the key results or advantages of each approach as well as their trade-offs and limitations.

3 Methodology

As mentioned above, the Pauli grouping problem can be solved by representing it as a graph colouring problem in which colours on the graph are interpreted as groupings of Pauli terms. Since the graph colouring problem requires adjacent vertices, which represent Pauli words that are QWC, to be of different colours, the complement graph must be used. The interpretation of two vertices that are connected in the complement graph is thus that they do not commute, so they should be of different colours. In this way, all vertices that share the same colour in the complement graph are all Pauli words that commute with each other. An example coloured complement QWC graph is shown below for illustration. It is the graph of the Hamiltonian for a hydrogen molecule, H_2, one of the Hamiltonians used in this project. From here onwards, unless specified otherwise, the term commuting will be used to refer to qubit-wise commuting and the graph we will be referring to is the complement graph.

This graph colouring representation of the Pauli grouping problem allows a QUBO formulation to be derived. The QUBO formulation is a way of representing the problem with binary variables in a quadratic cost function. This, in turn, is amenable to being solved using QAOA. Qiskit, a popular open-source software stack for quantum computing which allows programs to be run on IBM quantum computers, was used to implement and run QAOA. Due to the problem sizes encountered with the molecular Hamiltonians, the quantum programs were run on a simulator rather than an actual quantum computer. IBM's quantum computers are currently limited in the number of

Approach	Key result or advantage	Trade-offs and limitations
Uniform shot allocation[33]	Provides a simple baseline in which each Pauli string receives the same number of shots.	Does not account for differences in Pauli coefficients or variances, so it is generally not the most efficient way to reach a target precision.
Weighted and adaptive shot allocation[36,38]	Allocates more measurements to terms or groups that contribute more strongly to the estimator variance, reducing the total variance for a fixed shot budget.	Requires coefficient or variance information, and adaptive methods spend part of the shot budget estimating variances before reallocating measurements.
Hamiltonian truncation[24]	Removes Pauli terms with sufficiently small coefficients, reducing the number of terms that must be measured.	Introduces an approximation error controlled by the chosen tolerance, so overly aggressive truncation can reduce accuracy.
Shadow tomography and inference methods[3,33]	Infer properties of the quantum state from fewer measurements, potentially avoiding the cost of full tomography and reducing the number of shots needed for energy estimation.	Classical shadows require randomized measurements and post-processing, while neural network tomography requires model training and can introduce additional implementation complexity.
Unitary partitioning[19,33]	Rewrites the Hamiltonian as a weighted sum of near-minimum unitary operators, which can linearly reduce the number of terms to be measured.	Joint measurement can introduce covariance effects, and the additional circuit structure can add noise on NISQ devices.
QWC and GC Pauli grouping[35,14]	Groups mutually commuting Pauli words so multiple terms can be measured together; QWC can reduce measurements to roughly one third of the naive count, while GC can reduce scaling further in some cases from $O(N^4)$ to $O(N^3)$.	QWC is easy to implement but less general; GC can produce fewer groups but requires more complicated basis rotations, potentially involving non-local entangling gates and causing quantum circuit complexity to scale poorly ($O(N^2)$) in some cases.
Decomposed interaction methods[15,37]	Basis rotation grouping and Cartan decompositions rewrite the Hamiltonian so many terms become diagonal in a common basis, reducing the number of joint terms to measure.	They require orbital rotations or algebraic decompositions and can add circuit depth; their practical benefit depends on whether the reduced measurement count offsets this overhead.
Graph-based grouping heuristics[35]	Models Pauli grouping as a clique cover or graph colouring problem and applies classical heuristics such as greedy colouring, DSatur, RLF, and related methods.	Heuristics are practical and often effective, but they tend to introduce measurement co-variances.
Sorted insertion[7]	Orders Pauli strings by coefficient magnitude and inserts them into commuting groups to reduce covariance effects and directly target measurement cost.	Optimizes measurement precision more directly than group count alone, but performance depends on the chosen commutativity rule and still inherits grouping-related covariance concerns.

Table 1: Overview of VQE measurement-reduction approaches discussed in the literature review.

qubits they provide relative to the problem sizes encountered in this project, only having up to 156 qubits. Qiskit Aer is a simulator from the Qiskit Ecosystem that was used in this research project to fulfill this need. Only the smallest molecule, H_2, was run on an actual quantum computer, since it was the only molecule with a small enough Hamiltonian to be represented with the number of qubits available on IBM's quantum computers (using a free-to-use plan). To illustrate this, 45 qubits are required to represent the Hamiltonian for the hydrogen molecule using the one-hot encoding formulation, while only 30 qubits are required using the space-efficient embedding formulation. The next smallest molecule, LiH, requires 2500 qubits using one-hot encoding and 500 qubits using space-efficient embedding. However, IBM's freely available quantum computers have at most 156 qubits, so any molecule larger than H_2 cannot be run on real hardware. This also seems to be the limit on any of their QPUs, so there is indeed a limitation on the sizes of the problems that can be solved using the graph colouring approach used in this paper[17]. Additionally, even with the most performant quantum computer at D-Wave, a company that specializes in quantum annealing, the limit is 4,400 qubits, which is not enough to solve the optimization problem for the molecule BeH_2[8]. In order to solve these and more complex systems, different approaches would be required to use fewer qubits.

The methodology of our research involves using the one-hot encoding QUBO and space-efficient embedding formulations of the graph colouring problem in QAOA to attempt to find solutions that are closer to being optimal than have previously been discovered. The complete process, including the reformulation of the problem in the various formats, the generation of the data to be used in the optimization, the execution of the algorithms, and other details are described in further detail below.

As an overview of the steps involved, to evaluate the applicability of QAOA to the Pauli grouping problem, we construct a pipeline consisting of the following steps, which are depicted visually in the diagram in Figure 8 below:

1. Generation of Hamiltonian Pauli operator terms.
2. Commutativity graph generation from Pauli terms.
3. Formulation of the graph colouring problem as a one-hot encoding QUBO problem or space-efficient embedding.
4. Implementation of the QAOA algorithm to solve the graph colouring problem.
5. Execution of the algorithm on simulators and quantum hardware.

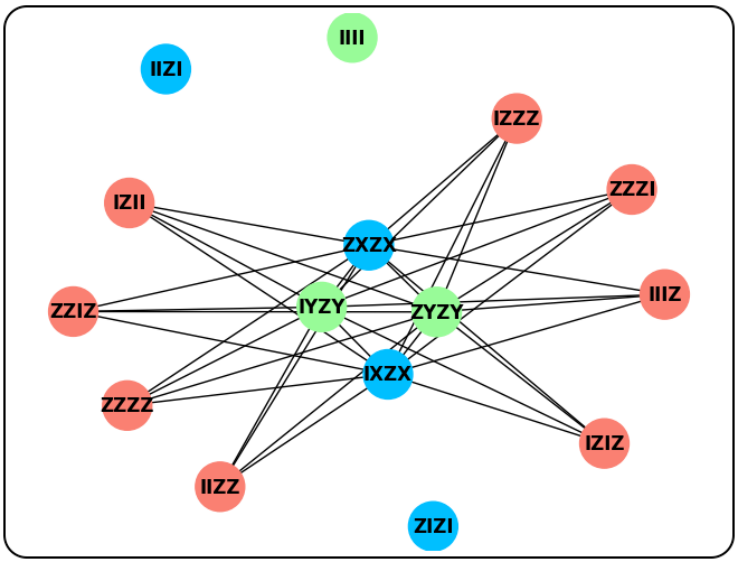


Fig. 7: The complement QWC graph for the Hamiltonian of the H_2 molecule, with the groups of mutually commuting terms coloured.

6. Analysis of the results and comparison to classical approaches.

First, the molecular Hamiltonian data was generated in order to create the graphs. The data chosen in this project is the same that is used in Verteletskyi et al.'s research[35]. In their paper, they apply classical heuristic graph colouring algorithms to the Pauli grouping problem for the Hamiltonians of certain molecules. The molecules they used include H_2 (hydrogen), LiH (lithium hydride), H_2O (water), BeH_2 (beryllium hydride), NH_3 (ammonia), and N_2 (nitrogen). Most of these molecules were included in this project, though not all due to the large size of some. Specifically, due to an intractable (using the proposed approach) number of Hamiltonian terms, the molecules NH_3 and N_2 were not included in this work, and some variations of the included molecules were not used either for the same reason. Table 2 below shows general information about the generated data for the chose molecules. N is the number of qubits used to represent the molecule (which is different from the number of qubits used in the QAOA implementation of the graph colouring problem) and also represents the number of Pauli operators in a single Pauli string. The Total column shows the number of Pauli terms in the Hamiltonian. Some molecules appear more than once. This is because there are different approximation methods that can be used to model molecules, some with more precision or accuracy. Verteletskyi et al. use the STO-3G and 6-31G basis sets to generate the Hamiltonians. There are also different ways of representing the resulting model as qubits. In the paper, they use the Bravyi-Kitaev (BK) and Jordan-Wigner (JW) transformations to do this. The same approaches were used in this research project to generate the data. The Basis and Transformation columns highlight this, showing the basis set and transformation used to generate each Hamiltonian. Note that parity for LiH means that a reduced setup with parity mapping and reductions was used, resulting in fewer terms than normal.

Systems	N	Total	Basis	Transformation
H_2	4	15	STO-3G	BK
LiH	4	100	STO-3G	Parity
H_2O	6	165	6-31G	BK
BeH_2	14	666	STO-3G	BK
BeH_2	14	666	STO-3G	JW

Table 2: Molecular Hamiltonian data used in this project. N is the number of qubits used to represent the molecule, and Total is the number of Pauli terms in the Hamiltonian.

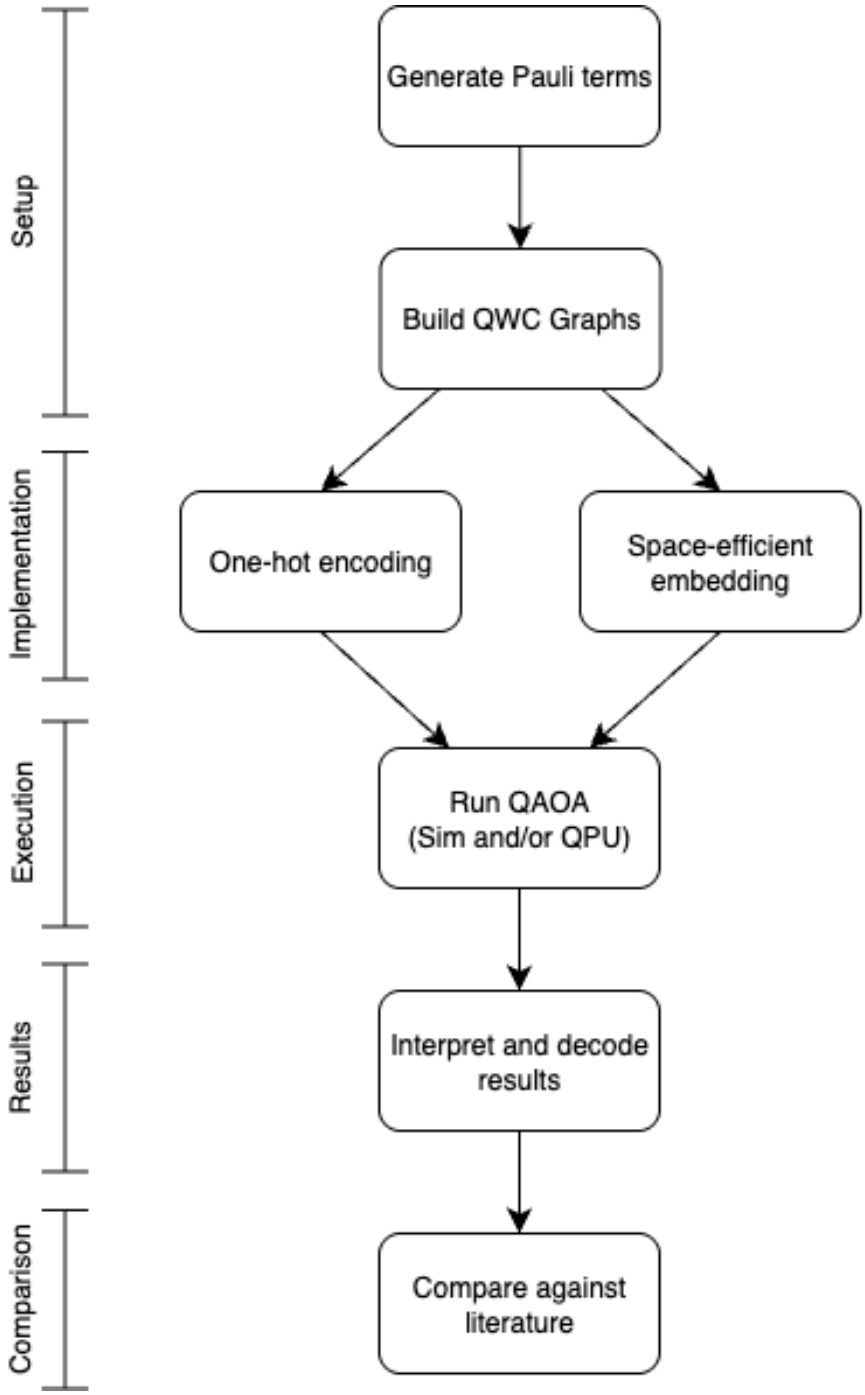


Fig. 8: Flowchart depicting the steps involved in the methodology of this research project.

Unfortunately, the Pauli terms in the Hamiltonians of the molecules are not available from Verteletskyi et al.'s work. However, a description of the methods used to devise their generation is given in supplemental material. Thankfully, the lab in which this research was performed is a quantum chemistry lab, so with some help from a peer, a program was created to generate the same number of Pauli terms for each molecule as shown in the table above.

The next step is to produce the qubit-wise commutativity graph from the generated Pauli words. This was straightforward to implement, requiring only a function to check that two vertices are QWC and a function to create an adjacency matrix using the QWC check. The complement graph can be directly created by using the negation of the QWC check to create an edge between two vertices, i.e. connect two vertices if they do not commute.

At this stage, the molecular Hamiltonians have been generated and the resulting Pauli words and their (non-) commutativity are represented in a graph. Next, the graph needs to be processed and coloured using QAOA. To get to this point, the QUBO formulation of the graph colouring problem must be encoded in a way that can be implemented using QAOA. Qiskit has a built-in class that implements the quantum algorithm. The process of using the `QAOA` class to solve the graph colouring problem involves a few steps. First, the graph colouring problem is provided as a `QuadraticProgram`. In Qiskit, the `QuadraticProgram` class can be used to represent an integer or binary variable problem which is quadratic in nature. Since any QUBO problem falls into this category, the QUBO formulation of the graph colouring problem (the one-hot encoding approach) can be stored as a `QuadraticProgram`. Next, an instance of the built-in `QAOA` class is created, and the desired backend must be provided in its initialization. In Qiskit, a backend is a processing device on which a quantum circuit is to be executed. These can be real quantum computers at IBM or a simulator processing circuits locally. Following this, the `QAOA` instance is wrapped in a `MinimumEigenSolver` class. This allows the translation from the mathematical representation of the problem as a `QuadraticProgram` in terms of binary variables to qubits and Hamiltonians used by `QAOA` and vice-versa. Finally, the `QuadraticProgram` created for the graph colouring problem is passed to the `MinimumEigenSolver`'s `solve` method, which executes the algorithm to solve the `QuadraticProgram` on the previously specified backend. The result of the optimization is returned, which is the solution that was found by the optimization process. It provides information such as the values for the variables in the `QuadraticProgram` and the cost function evaluation of the returned solution.

In addition to the one-hot encoding QUBO formulation of the graph colouring problem, the more resource-efficient space-efficient embedding was also implemented and run using QAOA. As mentioned above, this formulation is based on a different encoding of the problem which reduces the number of qubits required to represent it. This method was described in Tabi et al.'s work[31], and a review of it is given in the introduction. The space-efficient embedding is not a QUBO formulation since it includes terms that are higher order than quadratic. Tabi et al. directly implement the Hamiltonian representation of the space-efficient embedding instead of translating it to a QUBO formulation, which would require many auxiliary variables to quadratize the expression and thus more qubits. We follow the same approach in this work.

A few additional details and optimizations were implemented in the process of running the algorithms. First, the main variable that was passed into the algorithm was the number of colours k to be used in the graph colouring problem. This was done to investigate how the implemented quantum algorithms perform with different values of k. Multiple such values were tested for most molecules, starting from the number of groups found by the best-performing classical algorithm in Verteletskyi et al.'s work, then trying one less colour and one more. Since QAOA is probabilistic, even though it is possible to obtain the same solution as in [35], it is not guaranteed. Thus, trying $k+1$ colours allows the algorithm more freedom to find a solution (at the cost of a larger search space), while trying $k-1$ allows it to be tested on whether it can find a more optimal solution. Given the relative simplicity of hydrogen, only the values of k and $k+1$ from the paper [35] were tested for it. Due to the complexity of the BeH_2 molecule Hamiltonians, only the value of k from the paper was tested for it.

One optimization that was implemented was the use of custom mixers and initial states in the QAOA algorithm. By creating a custom or constraint-preserving mixer that allows the algorithm to explore the search space more efficiently by restricting it to valid solutions, it is possible to improve the results of the algorithm. When constraint-preserving mixers mix the probability amplitudes of different solutions, they do so only over valid solutions, preventing the optimizer from searching through invalid colourings. The constraints that were implemented through custom mixers in this project are the requirement for each vertex to be assigned exactly one colour in the case of the one-hot encoding implementation and the invalidity of non-existent colours in the case of the space-efficient implementation. Additionally, by creating a custom initial state that allows the algorithm to start from a more optimal point in the

search space than a uniform superposition of all states, namely by starting from a valid colouring, this can help further improve the performance of the algorithm.

Another one is the removal of vertices that fully commute with all other vertices in the graph from the optimization process. Since these vertices can be added to any group without affecting the validity of the solution, they do not need to be included in the optimization process. This reduces the size of the problem and thus the search space, which can help improve the performance of the algorithm and its tractability.

The classical optimizer used to optimize the parameters of the QAOA circuit was SPSA. SPSA is a stochastic optimization method that is well-suited for optimizing the parameters of quantum circuits, as it can handle noisy evaluations of the cost function and does not require gradient information. Additionally, it only requires two evaluations of the cost function per iteration, regardless of the number of parameters, which makes it efficient for optimizing quantum circuits with many parameters. It is the recommended optimizer for use in variational algorithms such as QAOA[16,20].

The execution of the program for simulating the quantum algorithms is summarized in the following steps:

1. Load the generated data for a given molecule (the Pauli terms in its Hamiltonian).
2. Create the QWC graph and its complement, but for performance reasons exclude any vertices that fully commute with all other vertices, since these can be added to any group.
3. Set up the `QAOA` instance. This includes the following steps:
   (a) create a custom mixer,
   (b) create a custom initial state,
   (c) chose the classical optimizer and set its parameters.
4. Then, initialize the `AerSampler` backend, choosing `matrix_product_state` as the method for simulating the quantum circuits, which uses much less resources than other methods.
5. The two different methods used, the one-hot encoding and the space-efficient embedding, are implemented and run separately.
   For the **one-hot encoding** method:
   (a) Create the `QuadraticProgram` for the graph colouring problem using the complement graph. This involves creating binary variables for each vertex and colour, and adding constraints to ensure that each vertex is assigned exactly one colour and that adjacent vertices do not share the same colour (refer to Equation 1 for more details).
   (b) Next, the `QAOA` instance is created and initialized with the desired backend and parameters, wrapped in an instance of the class `MinimumEigenSolver`, and the `QuadraticProgram` is passed to its `solve` method to execute the algorithm and obtain a solution.
   For the **space-efficient embedding** method:
   (a) Directly create the space-efficient cost Hamiltonian for the problem as described in Tabi et al.'s work[31] (see Equation 3 for more details).
   (b) Next, the `QAOA` instance is created and initialized with the desired backend and parameters and the cost Hamiltonian is passed to its `compute_minimum_eigenvalue` method to execute the algorithm and obtain a solution.
6. When the solution is obtained, the colours are decoded from the resulting bitstring, the Pauli groups are derived, and the removed vertices are added to the first group.

Since hydrogen was run on actual quantum hardware, the program for it was executed in a slightly different way. Similar steps were followed, with one key difference: instead of using the `AerSampler` backend, a real IBM QPU backend was used. The rest of the process was the same. Figure 9 shows a flowchart of the execution process for the quantum algorithms.

Due to high memory and processing requirements, these programs were run on high performance computing systems. These advanced computing resources are provided by the Digital Research Alliance of Canada (DRAC), the organization responsible for digital research infrastructure in Canada[1], and ACENET, its regional partner in Atlantic Canada. ACENET is funded by DRAC, the provinces of New Brunswick, Newfoundland and Labrador, Nova Scotia and Prince Edward Island, as well as the Atlantic Canada Opportunities Agency[2]. Using these systems was invaluable for this project, providing resources that would have otherwise been unavailable.

The results obtained from the execution of the algorithms were compared to the results found in Verteletskyi et al.'s research. The number of groups found by the algorithms was compared to the number of groups found by the classical algorithms in their paper. The performance of the one-hot encoding and space-efficient embedding methods was also compared to each other. These results are discussed in the next section.

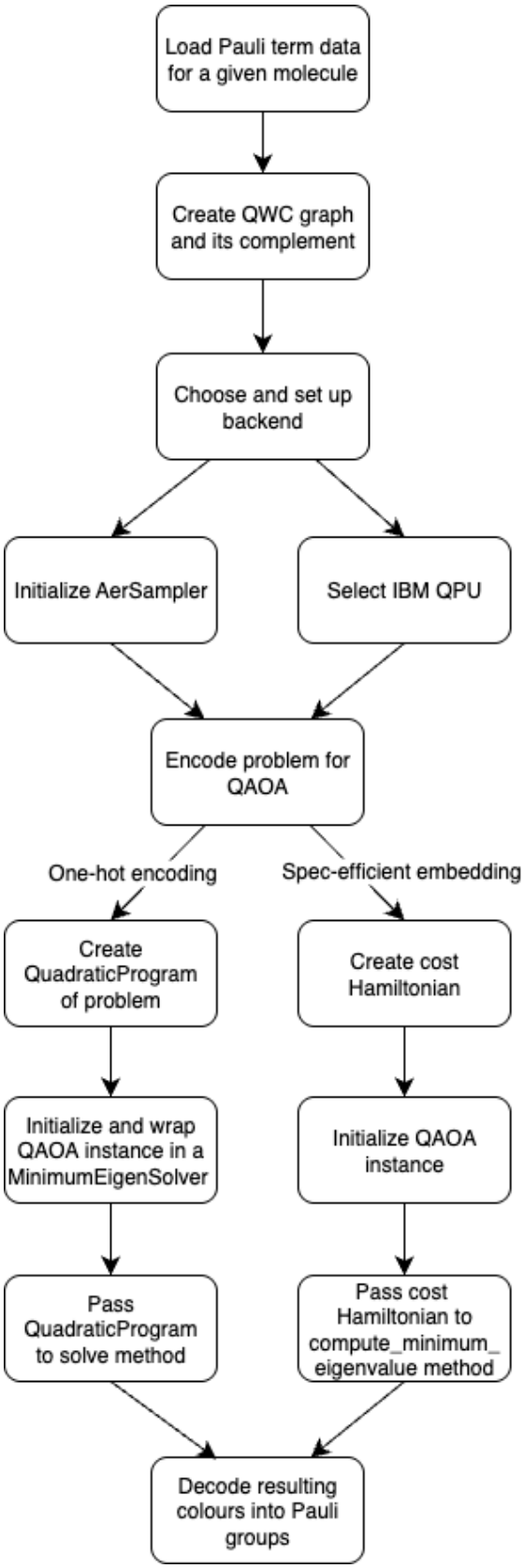


Fig. 9: Flowchart depicting the steps involved in the execution of the quantum algorithms.

4 Results

Table 3 below shows the specific molecules and the parameters of their Hamiltonians that were used in the execution of the algorithms. N represents the number of qubits used in the modelling of the molecule, or equivalently the size of each Pauli term, and Total (n) represents the total number of Pauli terms. The table also depicts the number of colours k used and whether the algorithms were run on a quantum computer or a simulator. In all cases, both the one-hot encoding and space-efficient embedding methods were used.

Systems	N	Total (n)	k	Simulation or QPU execution
H_2	4	15	3, 4	Simulation & QPU
LiH	4	100	24, 25, 26	Simulation
H_2O	6	165	33, 34, 35	Simulation
BeH_2	14	666	172	Simulation
BeH_2	14	666	172	Simulation

Table 3: Parameters of the Hamiltonians and graph colouring problems for the molecules used in the algorithms.

Table 4 shows the results found in [35] for the same molecules using classical algorithms. The first columns display the same information, though they add the basis sets and transformations used in the Systems column. The remaining columns following Total show the resulting number of minimum groups of completely connected (fully commuting) vertices found using various classical graph colouring algorithms. The algorithms they used are greedy colouring (GC), largest first (LF), smallest last (SL), DSatur, recursive largest first (RLF), Dutton and Brigham technique (DB), and COSINE. They also used two maximum clique search and removal algorithms to achieve a similar result which they refer to as Ramsey and BKT.

Systems	N	Total	GC	LF	SL	DSatur	RLF	DB	COSINE	Ramsey	BKT
H_2 (BK)	4	15	3	3	3	3	3	3	3	3	3
LiH (parity)	4	100	25	25	25	25	25	25	25	25	25
H_2O (6-31G, BK)	6	165	36	34	34	34	34	34	34	38	34
BeH_2 (BK)	14	666	175	172	172	172	172	172	176	190	175
BeH_2 (JW)	14	666	218	208	204	210	203	208	211	225	216
H_2O (BK)	14	1086	320	313	316	315	311	313	308	322	319
H_2O (JW)	14	1086	355	322	322	329	322	326	331	360	348

Table 4: Results from classical graph colouring algorithms applied to Pauli grouping problem for various molecules.

Of all the molecules used in this project, only H_2 was run successfully. The other molecules were too large for the QAOA simulations to complete within memory constraints or a reasonable time frame, even with the use of high performance computing resources. The simulations that were attempted for the larger molecules were allocated either 750 GB, 1125 GB, or 6000 GB of memory, and they ran for 16 or 24 hours before being terminated due to a crash or timeout. The crashes were either due to out of memory errors, memory allocation issues arising from insufficient memory, or low-level errors ostensibly emerging from the large burden on the simulator.

H_2 is the smallest Hamiltonian considered, so it acts as a feasibility and validation test for the QAOA implementation. Since Verteletskyi et al. report that H_2 can be grouped into 3 QWC groups, the simulations that use $k = 3$ test whether the algorithm can recover a known valid grouping, while those that use $k = 4$ test whether a slightly relaxed colouring problem is easier for QAOA to satisfy.

Table 5 shows the results obtained from the execution of the QAOA simulations for H_2. The columns show the problem encoding method used, the number of Pauli Hamiltonian terms, the number of colours k, and the number of qubits used in the encoding. The number of qubits in parenthese indicate the number of qubits needed after removing fully commuting terms before performing the optimization process. Since fully commuting terms can be added to any group, they can be omitted from the optimization, allowing the number of required qubits to be reduced. Additionally,

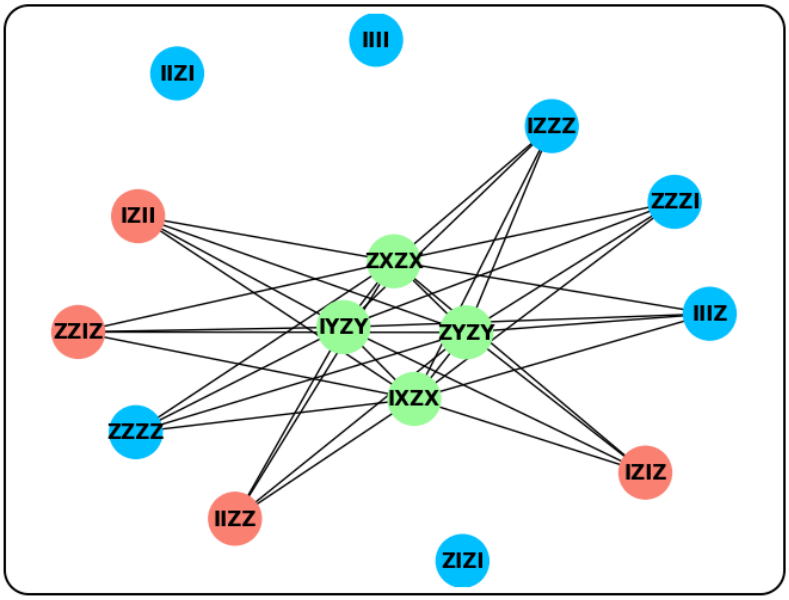
the table shows whether a valid colouring was found, the cost function evaluation of the returned solution, and the number of violations of the colouring constraints in the best solution, if known (if this information was not available, a value of N/A is given in the table). Note that the cost depends on the specific values chosen for the constraint weights C and D, so the costs are only comparable between instances in which the values of these variables are the same. A colouring was marked invalid if at least one edge (v, w) in the complement graph had both endpoints assigned the same colour. For the one-hot encoding, validity also requires each vertex to be assigned exactly one colour. The space-efficient embedding inherently satisfies this requirement from its construction, but an additional criterion it can violate is the use of invalid colour bitstrings in the case that k is not a power of 2. In summary, despite being different implementations, both the one-hot encoding and space-efficient embedding methods measure the validity of a given graph colouring using the same rules. The number of violations counts such conflicting edges and invalid colouring assignments. Figure 10 shows a comparison of invalid and valid colourings for the H_2 molecule. Figure 10a shows an example of an invalid colouring in which two pairs of adjacent vertices all share the same colour, namely the terms ZXZX, IYZY, ZYZY, and IXZX, and Figure 10b shows an example of a valid (and optimal) colouring. The graphs shown are the complement graphs, meaning edges indicate that terms do not commute and should not be grouped together, or alternatively that they should be coloured differently.

Systems	Method	Total (n)	k	Qubits	Valid colouring?	Cost	Violations
H_2	One-hot encoding	15	3	45 (36)	No	120	N/A
H_2	One-hot encoding	15	4	60 (48)	No	40	N/A
H_2	Space-efficient embedding	15	3	30 (24)	No	4	4
H_2	Space-efficient embedding	15	4	30 (24)	No	2	2

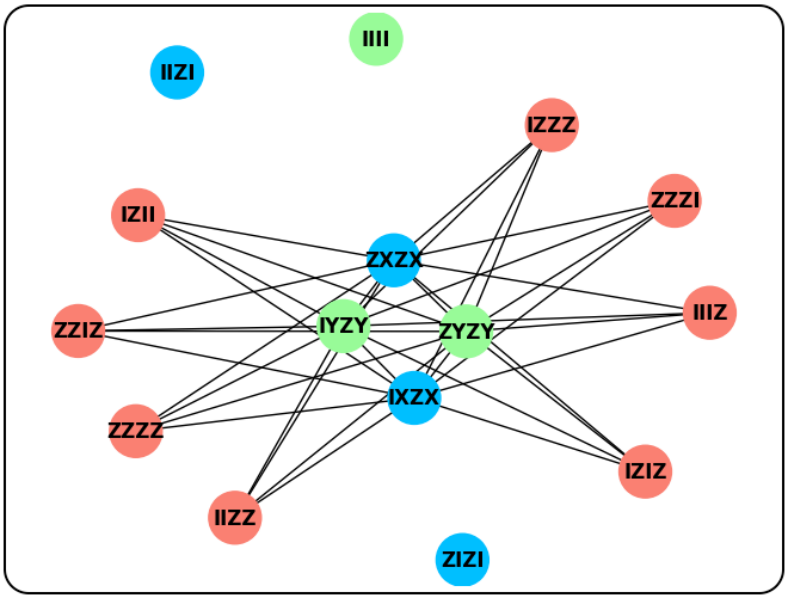
Table 5: Results for the simulations run on the H_2 molecule.

In the simulations, neither encoding produced a valid colouring for H_2. However, increasing the number of colours from $k = 3$ to $k = 4$ reduced the reported cost in both encodings. For the one-hot encoding, the cost decreased from 120 to 40, while for the space-efficient embedding both the cost and number of violations decreased from 4 to 2. This suggests that relaxing the constraint on the number of colours makes it easier to find more cost-effective solutions, although not enough for QAOA to produce a valid solution. This point is discussed further in the next section.

The QAOA instances run on real QPUs should be interpreted primarily as a hardware feasibility test rather than as evidence of algorithmic performance. Even for H_2, the encoded graph colouring problem required dozens of qubits, and the QAOA circuits had large depth and were more susceptible to noise. The IBM QPU backends that were used and their respective specifications are shown in Table 6. The table shows the backend name, processor, the number of qubits, the average two-qubit error rate per layered gate for a 100-qubit chain (layered meaning in a multi-qubit circuit), the T_1 and T_2 times, and the median readout error. The T_1 time represents the average duration a qubit remains in its excited state ($|1\rangle$) before decaying to its ground state ($|0\rangle$), whereas the T_2 time represents the average duration a qubit remains in its superposition state before being destroyed by environmental noise[18]. The resulting QPU measurements did not produce a valid colouring. This is consistent with the simulation results, but the hardware runs also show the additional difficulty introduced by device noise and physical qubit connectivity constraints. The results are depicted below in Table 7. In this case, the increase in the number of colours from $k = 3$ to $k = 4$ actually increased the cost, from 320 to 400. Despite the opposite trend in the simulations, this is not surprising given both the probabilistic nature of QAOA and the additional noise and errors introduced by the quantum hardware. Note that only the one-hot encoding results are shown, and no removal of fully commuting terms was performed before the optimization since the full problem size could fit within the available quantum hardware resources. No other methods or variations of the QAOA implementation were tested on QPUs because only a limited amount of compute time on IBM's quantum computers was available for this project, specifically 10 minutes.



(a) An example of an invalid solution for the H_2 molecule.



(b) An example of a valid (and optimal) solution for the H_2 molecule.

Fig. 10: Comparison of invalid and valid colourings for the H_2 molecule. In the invalid solution, two pairs of adjacent vertices share the same colour. The four edges that connect these vertices, which go around in a circle connecting the four green vertices, are the violations of the colouring constraints counted in the table above.

Although full QAOA solutions were not obtained for any of the larger molecules, namely LiH, H_2O, and BeH_2, the encoded problem sizes themselves are informative. Table 8 shows the total number of Pauli terms in the Hamiltonians of the molecules, the number of colours used in the simultations, and the resulting number of qubits required for the different encoding methods. The table also shows the memory and time limits that were allocated to the simulations for these larger molecules, which were not sufficient to obtain results. Regarding qubit count, LiH already requires roughly 2500 qubits under the one-hot encoding and 500 qubits under the space-efficient embedding. H_2O requires more than 5000 one-hot encoding qubits, while BeH_2 requires over 100,000. Even using the space-

Backend	Processor	Qubits	Two-qubit error rate	T_1 time	T_2 time	Readout error
`ibm_fez`	Heron r2	156	4.40×10^{-3}	147.85µs	106.95µs	1.43×10^{-2}
`ibm_kingston`	Heron r2	156	3.45×10^{-3}	273.55µs	126.97µs	9.70×10^{-3}
`ibm_marrakesh`	Heron r2	156	4.66×10^{-3}	197.63µs	101.64µs	1.11×10^{-2}

Table 6: QPU backends and corresponding specifications.

Systems	Method	Total (n)	k	Qubits	Valid colouring?	Cost
H_2	One-hot encoding	15	3	45	No	320
H_2	One-hot encoding	15	4	60	No	400

Table 7: Results for the H_2 molecule run on real quantum hardware.

efficient embedding, the problem sizes remain substantial, requiring at least 1000 qubits. These numbers show that the main obstacle is not only solution quality but the size of the encoded optimization problem. This exemplifies the challenges of scaling quantum algorithms to larger instances, especially for problems like graph colouring that have combinatorial complexity. This data and the resource requirements can be used as benchmarks for future attempts at solving the problem with quantum algorithms, as they indicate (less than) minimum resource requirements to run the algorithms for these instances.

Systems	Method	Total (n)	k	Qubits	Memory allocated	Time allocated
LiH	One-hot encoding	100	24	2400	750 GB	16h
LiH	One-hot encoding	100	25	2500	750 GB	16h
LiH	One-hot encoding	100	26	2600	750 GB	16h
LiH	Space-efficient embedding	100	24	500	750 GB	16h
LiH	Space-efficient embedding	100	25	500	750 GB	16h
LiH	Space-efficient embedding	100	26	500	750 GB	16h
H_2O	One-hot encoding	165	33	5445	1125 GB	24h
H_2O	One-hot encoding	165	34	5610	1125 GB	24h
H_2O	One-hot encoding	165	35	5775	1125 GB	24h
H_2O	Space-efficient embedding	165	33	990	1125 GB	24h
H_2O	Space-efficient embedding	165	34	990	1125 GB	24h
H_2O	Space-efficient embedding	165	35	990	1125 GB	24h
BeH_2	One-hot encoding	666	172	114552	6000 GB	24h
BeH_2	Space-efficient embedding	666	172	5328	6000 GB	24h

Table 8: Resource requirements for the larger molecules. The number of qubits required for the one-hot encoding is nk, while for the space-efficient embedding it is $n\lceil \log_2(k) \rceil$.

Overall, the H_2 experiments show that the implemented QAOA approach did not recover a valid known colouring for the smallest molecular Hamiltonian. Increasing k typically improved the objective values and reduced the number of violations but did not produce a valid solution. The larger molecular instances quickly exceeded practical simulation and QPU limits. These observations motivate the discussion of scalability, constraint enforcement, practicality, and the limitations of shallow QAOA in the following section, namely that QAOA is not currently a viable method for improving upon classical algorithms for the Pauli grouping problem, and that the representation of the problem in this manner is simply too expensive for current classical and gate-based quantum computers.

5 Discussion

We now return to the research question of interest: can QAOA be used to improve the results obtained in the Pauli grouping problem when compared with classical algorithms? The results obtained in this research project suggest that

the answer to this question is no, at least with the current state of technology. Even for small and relatively simple test cases, the QAOA implementation did not produce valid groupings, and for larger systems the resource requirements became prohibitive before useful comparisons could be made. Thus, not only did QAOA not improve upon classical heuristics, the representation of the optimization problem as a quantum algorithm became more expensive than the Pauli grouping problem itself. Due to the limited capabilities of current systems, the QAOA implementation we used is a reduced one. For example, only 3 repetitions of alternating cost and mixer layers were used in the QAOA circuit in this project. This is a small number of layers, and it is known that the performance of QAOA improves with more layers[9]. However, the number of layers also increases the depth of the circuit, which in turn increases the amount of noise and errors that can occur during the execution of the algorithm. Thus, there is a trade-off between the performance of the algorithm and its feasibility on current quantum hardware. Three layers were chosen as a compromise between the performance of the algorithm and the feasibility of running it on current quantum hardware and in simulations. With more layers, it may be possible to find better solutions to the problem, but it may not yet be possible to run such a circuit reliably. Consequently, our results do not necessarily reflect the true potential of the algorithm.

To elaborate on the infeasibility of the QAOA implementation for the Pauli grouping problem, it is important to separate the technological limitations from the algorithmic limitations. The technological limitations refer to the current state of quantum hardware and classical computing resources, while the algorithmic limitations refer to the inherent properties of the QAOA algorithm. Regarding the technological limitations, these include the limited number of qubits available on current quantum computers, the noise and errors that can occur during the execution of quantum circuits, the large circuit depth involved with additional cost and mixer layers in the algorithm, the immense scaling of simulating quantum systems on classical computers, and the low entanglement requirement for the matrix product state simulation method to remain manageable. The algorithmic limitations include the fixed value of k, the probabilistic nature of the algorithm, constraint penalties potentially failing to guide the optimization process toward valid solutions, classical optimizer choice with respect to the optimization landscape, shallow circuit depth being insufficient, feasible colourings occupying a potentially small portion of the search space, and the existence of barren plateaus in the optimization landscape encumbering parameter optimization.

Most results obtained using QAOA did not come particularly close to finding valid solutions. The invalid solutions returned by the algorithm violated the constraints of the problem, namely the constraint that adjacent vertices in the graph must be of different colours. The number of violations of this constraint was often not very high with respect to the total number of edges in the graph, but it was still significant enough to make the cost function evaluation high for the returned solutions. Additionally, grouping non-commuting Pauli terms together has a knock-on effect on the groupings of other terms. As the algorithm explores the search space near such an invalid solution, there are conflicting forces due to the interaction between other terms in the Hamiltonian with the non-commuting terms. This can cause terms that can be measured together to be separated into different groups, which could result in more groups than necessary. Thanks to the use of custom mixers and initial states, the algorithm was able to find solutions that satisfied the one colour per vertex constraint. Without these improvements, the algorithm could have returned solutions that violated this constraint as well, which would have resulted in even higher cost function evaluations and worse solutions. An interesting observation is that increasing the value of k reduced the overall cost function evaluation of the returned solutions. The intent with varying k was to provide the optimization process with more colours, giving the algorithm has more freedom to find a solution that satisfies the constraints of the problem and also more grouping combinations from which to pick, which would hopefully lead to lower cost function evaluations. However, this is actually counterproductive for two main reasons. First, this increases the search space, which can make it more difficult for the algorithm to find good solutions. Second, there is a logical explanation to why the reduction in the cost function evaluation occurs. With more colours, the constraints are easier to satisfy. Notice that the constraints don't depend on the number of colours, and nor does the cost. They penalize invalid colourings, not the use of more colours. At the extreme, if we pick the maximum number of colours, nk in the one-hot encoding method and $n\lceil \log_2(k) \rceil$ in the space-efficient embedding approach, the problem becomes trivial, and assigning one colour to each vertex results in a cost of 0. Furthermore, increasing k also increases the number of groups, which is the opposite of what we are trying to achieve.

Now we compare in more detail the one-hot encoding and space-efficient embedding methods used in this project. As previously mentioned, the one-hot encoding method is a straightforward way to represent the graph colouring problem as a QUBO problem, which can then be solved using QAOA. However, it requires many qubits, which makes it impractical for larger graphs. The space-efficient embedding method, on the other hand, uses a different

encoding that reduces the number of qubits required to represent the problem. With this, however, comes the cost of a more complex cost Hamiltonian that includes higher-order terms, which can make it more difficult to implement and optimize. The constraint-preserving mixer is necessarily also more complex given the binary encoding of the colours. The more complicated cost Hamiltonian and mixer increase circuit depth, which can lead to more noise and errors during the execution of the algorithm. Additionally, the more complex cost Hamiltonian can make it more difficult for the classical optimizer to find good parameters for the QAOA circuit. These factors can reduce the performance of the algorithm. Therefore, while the space-efficient embedding improves representational cost, it does not remove the underlying combinatorial difficulty; it shifts the bottleneck from the number of qubits toward Hamiltonian and mixer complexity, circuit depth, and the optimizer's ability to find feasible colourings.

This project focused on optimizing the number of QWC groups of Pauli terms to tackle measurement issues encountered in VQE. It did not consider the total variance or shot count required to estimate the Hamiltonian expectation value to a target precision. As discussed in the literature review, grouping can introduce covariance effects, and some approaches directly optimize measurement allocation rather than group count. Therefore, even successfully applying QAOA to the graph colouring problem in the context of Pauli grouping would only address one component of the VQE measurement bottleneck. A complete comparison would need to evaluate the number of groups, the resulting estimator variance, the shot budget required for a target error ϵ, and any additional circuit depth required by the measurement scheme. Such a comprehensive analysis is beyond the scope of this project but is important to consider in future work to improve upon current results in the literature.

Another thing to note is that our current method of implementing the Pauli grouping problem as a quantum circuit implementing QAOA does not adjust the number of colours automatically. A single evaluation of the problem must be performed for a specific number of colours. Thus, if the number of colours that can be used to colour a given graph is not known or if no approximations are known from which to begin, the algorithm may need to be run multiple times for varying numbers of colours. In our case, the data from Verteletskyi et al.'s results was available, so we could start with the minimum number of groups found for each molecule and work from there. Note that it may be possible in some cases to estimate the minimum number of colours needed for a given graph. For example, if the average graph connectivity is above a certain threshold that grows asymptotically as $2k\ln k$ for k colours, then the graph cannot be coloured with k colours[23]. Knowledge of such properties of the graph could be used to estimate the minimum number of colours needed, which would reduce the number of times the algorithm needs to be run. However, even with such information, the structural weakness of the QAOA implementation remains: multiple runs may be needed to find the proper value of k. To state it more clearly, classical approaches tackle the graph colouring problem by starting with few colours and increasing it until a valid colouring is found, whereas the QAOA implementation answers the question of whether a valid colouring exists for a specific number of colours. Thus, the QAOA implementation does not directly find the minimum number of colours needed to colour the graph, and multiple runs may be needed to find this number, multiplying the already high implementation cost. It changes the optimization problem from a minimization problem to a decision problem, further weakening the practicality of this approach.

However, the results do demonstrate that with the current state of technology, the application of QAOA to the Pauli grouping problem is not a viable approach for finding solutions similar to or better than those found by classical algorithms. None of the successful runs of the algorithm produced a completely valid solution, they failed to find known numbers of groupings for certain molecules, and no more optimal solutions were found than those found using classical means. Additionally, the resources required to run the classical heuristic algorithms, in terms of both time and memory, are undoubtedly much smaller than those required to run the simulations of the QAOA algorithm, and potentially less than those required to run the algorithm on real quantum computers. Assuming this is true, this further supports the conclusion that QAOA should not be used to solve the Pauli grouping problem, given the current state of technology. QAOA's probabilistic nature also means that it is not guaranteed to find the optimal solution, or even a valid solution, in a single run. Thus, multiple runs may be needed to find a good solution, which further increases the resources required to use this algorithm. The heuristics used in the classical algorithms, on the other hand, are deterministic, relatively fast, and have been shown to produce good results for the Pauli grouping problem, while also framing the problem as a minimization problem rather than a decision problem, which is more practical for finding the optimal number of groups.

6 Conclusion

6.1 Summary

This research project investigated the use of the Quantum Approximate Optimization Algorithm to solve the Pauli grouping problem, a problem that arises in the context of variational quantum algorithms such as the Variational Quantum Eigensolver. The Pauli grouping problem can be represented as a graph colouring problem, and we implemented QAOA to solve it using a one-hot QUBO encoding and a space-efficient embedding. We generated the Hamiltonians for several molecules, created the corresponding graphs, and ran the QAOA instances on both quantum hardware and a classical simulator to find groupings of Pauli terms. We compared our results to those obtained from classical heuristic algorithms in previous research performed by Verteletskyi et al. We also discussed the challenges encountered in simulating quantum algorithms for large problem sizes and the limitations of current quantum hardware. The experiments showed that, although the mapping is theoretically natural, the resulting optimization problems are too large for current QPUs and very expensive to simulate classically. In the completed runs, QAOA did not find valid groupings that matched or improved upon known classical heuristic results. These results suggest that, with present hardware and shallow QAOA circuits, this approach is not competitive with classical graph colouring heuristics for Pauli grouping.

Despite the promising nature of quantum computing, current quantum computers are not capable of running the QAOA algorithm proposed in this paper for the Pauli grouping problem, even for the Hamiltonians of relatively small molecules. The problem sizes are simply too large, even with the space-efficient embedding of the graph colouring problem. Consequently, the number of qubits required to perform the optimization procedure presented in this work on most molecules exceeds the available number of qubits on any quantum computer. As mentioned above, this is why we turned to classical simulations, with the intent to simulate the QAOA circuit and obtain results similar to what might emerge from a true quantum application of this problem. Unfortunately, due to the immense scaling of simulating quantum systems and quantum computing, this also proved to be very challenging, even for supercomputers such as those available through DRAC. The algorithm could only be simulated for the simplest molecule, namely hydrogen. For the larger molecules, the simulations could not be completed within a reasonable amount of time or with the available resources. This is a clear demonstration of the challenges that arise when trying to apply quantum algorithms to real-world problems with current technology. The limitations of current quantum hardware and the challenges of simulating quantum algorithms on classical computers are significant barriers to the practical application of quantum algorithms for problems such as the Pauli grouping problem.

Despite the challenges that were encountered, our approach does give insight into the means by which the problem could be solved and the limitations current computing systems, both classical and quantum in nature, impose on them. A repetition of this experiment would need to be performed at a time when either there are sufficient classical resources to simulate the QAOA implementation or a quantum computer with enough qubits exists, assuming the Pauli grouping problem is still relevant at that time.

6.2 Future Work

Several optimizations were attempted in this project to try to improve either the efficiency, tractability, and results of the QAOA implementation of the Pauli grouping problem. The space-efficient embedding was used to reduce the number of qubits required in the QAOA simulations from nk to $n\log_2(k)$ for a problem with n vertices and k colours. This approach significantly reduced the resource requirements for the simulations, though it did not allow for the successful simulation of larger molecules such as LiH, H_2O, and BeH_2.

Additionally, several small improvements were implemented by tuning parameters and changing certain values in the Qiskit classes being used such as the parameters passed to the QAOA instance and to the Qiskit Aer simulator. Despite having implemented some of these refinements, there may be more room for improvement. A limited amount of effort was spent exploring the potential gains of modifying these parameters, so pursuing such an investigation could prove beneficial. However, unless the problem is simplified by orders of magnitude of complexity, these approaches will not enable the simulation of larger molecules. Parameter tuning could have potentially allowed the simulation of some of the molecules that failed to run in our case by allowing deeper quantum circuits, more elaborate computations, or much less memory usage, but it would not prove useful in general.

We would like to mention a few potential avenues of research that could be pursued in the future to try to improve upon the methods used and the results obtained in this project and in the literature. First, various aspects of

the QUBO and QAOA implementations could be investigated. For example, the penalty weights C and D used in the one-hot encoding method could be tuned to try to improve the performance of the algorithm, and the parameters of the QAOA circuit such as the optimizer used could be tuned as well. Different constraint-preserving mixtures could be designed and implemented, and the initial state could be changed to try to improve the results. One improvement that should be investigated is an increase in the number of repetitions p of alternating cost and mixer layers in the QAOA circuit in a more ideal setting free of noise and errors. Furthermore, warm starting the optimization process with a good initial guess for the parameters could potentially lead to better results. For example, the parameters could be initialized based on the results of classical preprocessing, or an initial state could be prepared based on a known good solution from a classical algorithm. This could potentially help the optimization process find better solutions more quickly.

This project focused on a QAOA implementation of the graph colouring problem, though other quantum optimization algorithms could be explored as well. Further gate-based quantum algorithms could be explored, but other types of quantum computing could be utilized as well. Tabi et al. use a quantum annealer to solve the graph colouring problem[31]. Quantum annealing is a different approach to quantum optimization, and it would be interesting to compare the results obtained from a quantum annealing approach to those obtained from a gate-based approach such as QAOA. There could be further approaches to be explored through the use of quantum annealing, so this is another avenue of research that could be pursued in the future.

A further aspect to consider in future work is combining various strategies to tackle the real problem of VQE measurement optimization. As mentioned in the literature review, the Pauli grouping problem is only one component of the measurement bottleneck in VQE. Thus, methods should be explored that combine approaches to optimize the number of groups, the resulting estimator variance, and the shot budget required for a target error ϵ, as well as taking any additional restrictions or requirements into consideration, leading to a more holistic approach. Along the same lines, different approaches to these various components could be investigated and compared to find the best combination of methods for optimizing VQE measurements. For example, QWC and general commuting groupings could be compared, and different approaches to optimizing measurement allocation could be explored and compared as well.

An adaptive search over the number of colours k would lead to an important change in the algorithm's meaning and use. Instead of acting as a decision problem answering whether a given graph can be coloured using k colours, being able to minimize the value of k as part of the algorithm would transform the algorithm into a true optimization algorithm. A potential approach occurred to us during a meeting in which we discussed the QUBO formulation of the graph colouring problem. One-hot encoding the colours is a relatively straightforward mathematical implementation of the problem, so we were considering what other terms could be added to the cost function to add additional constraints or to encourage certain outcomes. We tried to produce a term in the expression that would encourage the use of fewer colours by penalizing the use of more colours than are necessary. This would remove the need to run the algorithm multiple times for different values of k to find the minimum number of colours needed to colour the graph.

In our discussion, we came up with the concept of a term that encourages a smaller number of colours. It involves summing over the colours and pairs of vertices to count the number of pairs of vertices that are assigned the same colour:

$$-A \sum_{i} \sum_{v,w} x_{v,i} x_{w,i}.$$

Here, A is the penalty term's weight. This term encourages the use of fewer colours by assigning a negative cost to pairs of vertices that are assigned the same colour. The more pairs of vertices that are assigned the same colour, the lower the cost. The more colours that are used, chances are that fewer pairs of vertices are assigned the same colour, and thus the higher the cost. The fewer colours that are used, chances are that more pairs of vertices are assigned the same colour, and thus the lower the cost. It is important to carefully balance the penalty weight A to ensure that this term does not overpower the constraint terms in the cost function, which could lead to invalid colourings being favored over valid ones. Proper choice of A compared to C and D would be crucial to encourage the use of fewer colours while still ensuring that the algorithm prioritizes finding valid colourings.

Another potential issue is that the new term could create a more complex cost Hamiltonian, require more elaborate custom mixers, or create a more complex optimization landscape, all of which could make the algorithm more

difficult to implement and optimize. Thus, while this approach could potentially remove the need for multiple runs to find the minimum number of colours needed, it may also introduce new challenges that need to be addressed.

7 Note on LLM Usage

This project involved the use of large language models (LLMs), specifically ChatGPT and GitHub Copilot, to assist with various aspects of the research and writing process. The written components of this report that were generated with the help of LLMs include the table summary of the literature review, the glossary, the addition and truncation of table and figure titles in the table of contents, and the generation of some figures involving quantum circuits. These were reviewed and edited to ensure accuracy. The above LLMs were used to increase the development speed to complete the project in time. The code that was generated with the help of LLMs typically involved small, straightforward, or tedious snippets of code, and some other more complicated code to either improve the program's performance or to implement a specific feature. Additionally, it was also used to change an existing part of the program to implement it in a different way for quick prototyping and testing. The specific code that was generated includes the code to generate the Hamiltonians for the molecules; the process that removes fully-commuting terms from the optimizatin process and adds them back in afterward; some flag features to make the program more flexible; the copying of a feature implemented in one git branch to another; the implementation of the constraint-preserving mixers and the initial states used in the QAOA circuit; some of the setup in the cost Hamiltonians of the problems such as the `QuadraticProgram` pipeline and the space-efficient embedding method; and some of the colour decoding and Pauli grouping steps in the post-processing.

8 Appendix

This project involved many programs to deal with the various requirements of the research problem. A program was needed to generate the Hamiltonians and subsequent Pauli terms for the molecules. Another was required to generate the graphs from these Pauli terms and perform the QAOA simulation on them. Following this, various scripts were necessary to run the QAOA simulation for each molecule. All of these programs can be found on my personal GitHub page[5] (currently the two methods, one-hot encoding and space-efficient embedding, are in the branches `one-hot-encoding` and `see-no-qubo`, respectively), and they will also be added to QuNB's GitHub page[29]. The following code snippets include the major components of the programs for quick reference.

```
def graph_colouring_qubo_qp(A: list[list[int]], k: int, C: int, D: int,
    name:str="graph_colouring_qubo"):
    """
    Build the QuadraticProgram for

        C * sum_v (1 - sum_i x[v,i])^2
      + D * sum_{v,w} sum_i A[v][w] x[v,i] x[w,i]

    Parameters
    ----------
    A : list[list[float]] or 2D array
        n x n adjacency matrix.
        For an undirected graph, A should typically be symmetric with zeros on the diagonal.
    k : int
        Number of colours.
    C, D : float
        Penalty weights.

    Returns
    -------
    qp : QuadraticProgram
    """
```

```python
    n = len(A)
    qp = QuadraticProgram(name)

    # Create binary variables x_{v,i}
    x = {}
    for v in range(n):
        for i in range(k):
            var_name = f"x_{v}_{i}"
            qp.binary_var(var_name)
            x[(v, i)] = var_name

    linear = defaultdict(float)
    quadratic = defaultdict(float)

    # Constant part from C * sum_v 1
    constant = C * n

    # First term:
    # C * sum_v (1 - sum_i x_{v,i})^2
    #
    # For binary x:
    # (1 - sum_i x_i)^2 = 1 - sum_i x_i + 2 * sum_{i<j} x_i x_j
    for v in range(n):
        # linear: -C * x_{v,i}
        for i in range(k):
            linear[x[(v, i)]] += -C

        # quadratic: +2C * x_{v,i} x_{v,j} for i<j
        for i in range(k):
            for j in range(i + 1, k):
                quadratic[(x[(v, i)], x[(v, j)])] += 2.0 * C

    # Second term:
    # D * sum_{v,w} sum_i A[v][w] x_{v,i} x_{w,i}
    #
    # Since QuadraticProgram stores each pair once, we combine (v,w) and (w,v).
    # For symmetric A, this reproduces the full double sum exactly.
    for v in range(n):
        for w in range(v + 1, n):
            edge_weight = A[v][w] + A[w][v]
            if edge_weight != 0:
                coeff = D * edge_weight
                for i in range(k):
                    quadratic[(x[(v, i)], x[(w, i)])] += coeff

    qp.minimize(
        constant=constant,
        linear=dict(linear),
        quadratic=dict(quadratic),
    )
    return qp
```

Listing 1: The code used to construct the QUBO QuadraticProgram for the one-hot encoding method.

```python
def run_qaoa(qp: QuadraticProgram, reps: int, n: int, k: int, optimization_level: int, service: QiskitRuntimeService | None, simulation: bool = True):
    algorithm_globals.random_seed = 42
    qubo = QuadraticProgramToQubo().convert(qp)
    result = None
    optimizer, mixer_circuit, initial_point = build_qaoa_components(n, k, reps)
    initial_state_circuit = initial_state(n, k)
    if simulation == False:
        if service is None:
            raise ValueError("A QiskitRuntimeService instance is required when simulation=False.")

        backend = service.least_busy(simulator=False, operational=True, min_num_qubits=n * k)
        print(backend)

        sampler = Sampler(mode=backend)

        # Build the hybrid QAOA solver
        pm = generate_preset_pass_manager(
            backend=backend,
            optimization_level=optimization_level,
            translation_method='translator',
            routing_method='sabre'
        )
        qaoa = QAOA(
            sampler=sampler,
            optimizer=optimizer,
            reps=reps,
            initial_state=initial_state_circuit,
            mixer=mixer_circuit,
            initial_point=initial_point,
            pass_manager=pm
        )

        # Wrap QAOA as an optimizer for QuadraticProgram
        solver = MinimumEigenOptimizer(qaoa)

        # Solve
        result = solver.solve(qubo)

    else:
        backend_options = dict(
            method="matrix_product_state",
            max_parallel_threads=64,
            matrix_product_state_max_bond_dimension=32,
            matrix_product_state_truncation_threshold=1e-8,
            max_memory_mb=0,
        )

        qaoa = QAOA(
            sampler=AerSampler(
                run_options={'shots': 256},
                backend_options=backend_options
            ),
```

```
            optimizer=optimizer,
            reps=reps,
            initial_state=initial_state_circuit,
            mixer=mixer_circuit,
            initial_point=initial_point,
        )

        optimizer = MinimumEigenOptimizer(qaoa)
        result = optimizer.solve(qubo)
    return result, qubo
```

Listing 2: The code used to run the QAOA algorithm for the one-hot encoding method.

```
def space_efficient_coloring_hamiltonian(graph, k, invalid_penalty=0.0,
↪   conflict_penalty=1.0):
    if not isinstance(graph, dict):
        graph = adjacency_matrix_to_list(graph)

    n = len(graph)
    m = color_bit_width(k)
    n_qubits = n * m

    H = SparsePauliOp.from_list([("I" * n_qubits, 0.0)])

    # When the mixer preserves the valid-color subspace these terms are
    # unnecessary, but keeping them optional makes the Hamiltonian usable
    # outside the constrained ansatz as well.
    if invalid_penalty != 0.0 and k < 2**m:
        for v in range(n):
            qubits = [v * m + i for i in range(m)]

            for c in range(k, 2**m):
                P = projector_term(n_qubits, qubits, color_to_bits(c, m))
                H += invalid_penalty * P

    # --- EDGE PENALTY ---
    for u, v in graph_edges(graph):
        qubits_u = [u * m + i for i in range(m)]
        qubits_v = [v * m + i for i in range(m)]

        for c in range(k):
            bits = color_to_bits(c, m)
            P_u = projector_term(n_qubits, qubits_u, bits)
            P_v = projector_term(n_qubits, qubits_v, bits)
            H += conflict_penalty * (P_u @ P_v)

    return H.simplify()
```

Listing 3: The code used to construct the cost Hamiltonian for the space-efficient embedding method.

```
def projector_term(n_qubits, qubit_indices, bitstring):
    """
```

```
    Build projector onto a specific bitstring using Z operators.
    """
    terms = []

    for signs in itertools.product([0, 1], repeat=len(bitstring)):
        coeff = 1.0
        pauli = ["I"] * n_qubits

        for i, (bit, s) in enumerate(zip(bitstring, signs)):
            q = qubit_indices[i]

            if s == 0:
                coeff *= 0.5
            else:
                coeff *= 0.5 * (1 if bit == 0 else -1)
                # SparsePauliOp labels are ordered q_{n-1} ... q_0.
                pauli[n_qubits - 1 - q] = "Z"

        terms.append(("".join(pauli), coeff))

    return SparsePauliOp.from_list(terms)
```

Listing 4: The code used to construct the projector terms for the cost Hamiltonian.

```
def run_qaoa_H(H, n, k, reps=2, service: QiskitRuntimeService | None = None,
 ↪  optimization_level: int = 2, simulation: bool = True):
    algorithm_globals.random_seed = 42
    optimizer, initial_state, mixer, initial_point =
     ↪  build_binary_qaoa_components(n, k, reps)
    validate_direct_qaoa_dimensions(H, n, k, initial_state=initial_state,
     ↪  mixer=mixer)
    print('Number of qubits:', H.num_qubits)

    if simulation == False:
        if service is None:
            raise ValueError("A QiskitRuntimeService instance is required
             ↪  when simulation=False.")

        backend = service.least_busy(simulator=False, operational=True,
         ↪  min_num_qubits=H.num_qubits)
        print(backend)

        sampler = Sampler(mode=backend)
        pm = generate_preset_pass_manager(
            backend=backend,
            optimization_level=optimization_level,
            translation_method='translator',
            routing_method='sabre'
        )

        qaoa = QAOA(
            sampler=sampler,
            optimizer=optimizer,
            reps=reps,
            pass_manager=pm,
```

```
            initial_state=initial_state,
            mixer=mixer,
            initial_point=initial_point,
        )
        return qaoa.compute_minimum_eigenvalue(H)

    if H.num_qubits <= 20:
        qaoa = QAOA(
            sampler=StatevectorSampler(),
            optimizer=COBYLA(maxiter=200),
            reps=reps,
            initial_state=initial_state,
            mixer=mixer,
            initial_point=initial_point,
        )
        return qaoa.compute_minimum_eigenvalue(H)

    backend_options = dict(
        method="matrix_product_state",
        max_parallel_threads=64,
        matrix_product_state_max_bond_dimension=32,
        matrix_product_state_truncation_threshold=1e-8,
        max_memory_mb=0,
    )

    spsa = SPSA(
        maxiter=300,
        learning_rate=0.05,
        perturbation=0.1,
        blocking=True,
        allowed_increase=0.01
    )

    qaoa = QAOA(
        sampler=AerSampler(
            run_options={'shots': 256},
            backend_options=backend_options
        ),
        optimizer=spsa,
        reps=reps,
        initial_state=initial_state,
        mixer=mixer,
        initial_point=initial_point,
    )

    return qaoa.compute_minimum_eigenvalue(H)
```

Listing 5: The code used to run the QAOA algorithm for the space-efficient embedding method.

9 Glossary

This glossary emphasizes the technical terms most relevant to the project implementation and analysis.

Backend The device or simulator on which a quantum circuit is executed. A backend may refer to an IBM quantum processing unit (QPU) or to a local/classical simulator such as Qiskit Aer.

Basis set A set of functions used to build molecular orbitals when generating a molecular Hamiltonian. This project uses basis sets such as STO-3G and 6-31G to match the molecular Hamiltonian data studied in previous work.

Bravyi-Kitaev (BK) transformation A mapping from fermionic molecular operators to qubit operators.

Classical optimizer The classical part of a hybrid quantum-classical algorithm that updates circuit parameters based on measured or simulated cost values. QAOA uses a classical optimizer to choose parameters that reduce the chosen cost function.

Classical grouping heuristic A conventional algorithm used to approximate the Pauli grouping problem without running a quantum optimization routine.

Clique A set of vertices in a graph where every pair of vertices is connected by an edge. In the QWC graph, a clique corresponds to a group of mutually qubit-wise commuting Pauli words that can be measured together.

Complement graph A graph formed by replacing non-edges with edges and edges with non-edges. This project colours the complement of the QWC graph so that non-commuting Pauli words are forced to receive different colours.

Cost function The objective function being minimized by an optimization algorithm. In this project, the cost function penalizes invalid graph colourings, such as assigning adjacent vertices the same colour or failing to assign a vertex exactly one colour.

Cost Hamiltonian The Hamiltonian whose expectation value represents the optimization cost in QAOA. Lower-energy states correspond to better candidate solutions for the encoded graph colouring problem.

Expectation value The average value expected from repeated measurements of an observable. VQE requires estimating the expectation value of the molecular Hamiltonian.

Fully commuting vertex A vertex representing a Pauli word that commutes with every other Pauli word under the chosen commutativity rule.

Graph colouring problem The problem of assigning colours to vertices so that adjacent vertices do not share a colour.

Hamiltonian An operator representing the total energy of a quantum system. Hamiltonians are decomposed into sums of Pauli terms so their expectation values can be estimated on quantum hardware or simulators.

Initial state The starting quantum state prepared before QAOA applies its alternating cost and mixer operations. A custom initial state can bias the algorithm toward more promising regions of the solution space.

Jordan-Wigner (JW) transformation A mapping from fermionic molecular operators to qubit operators. It is another transformation used to generate qubit Hamiltonians for comparison with the BK transformation.

Matrix product state (MPS) A tensor-network representation of a quantum state that can make some circuit simulations more memory-efficient.

Measurement group A set of Pauli words that can be measured in the same tensor product basis. Fewer measurement groups generally means fewer distinct measurement circuits are needed to estimate the Hamiltonian expectation value.

Measurement shot One repeated execution and measurement of a quantum circuit. Many shots are needed to estimate probabilities and expectation values accurately.

Minimum clique cover (MCC) problem The problem of covering all vertices of a graph using the fewest possible cliques.

MinimumEigenSolver A Qiskit optimization interface that standardizes how quantum algorithms calculate the minimum eigenvalue of a given operator. It orchestrates the interaction between the quantum circuit, the classical optimizer, and a backend.

Mixer The part of a QAOA circuit that moves probability amplitudes between candidate solutions. A custom mixer can restrict the search to valid or more useful regions of the solution space.

Molecular Hamiltonian The Hamiltonian describing the electronic structure and energy of a molecule.

NISQ device Noisy Intermediate-Scale Quantum device: a current or near-term quantum computer with a limited number of noisy qubits. NISQ limitations restrict the size and depth of quantum circuits/algorithms that can be run on real hardware.

One-hot encoding An encoding where each vertex-colour choice (in the case of graph colouring) is represented by a separate binary variable, and exactly one variable should be active for each vertex.

Pauli grouping problem The optimization problem of partitioning Pauli words into the fewest possible measurement groups, subject to the requirement that all words in a group can be measured together.

Pauli operator One of the single-qubit operators X, Y, or Z, often considered alongside the identity operator I. Pauli operators form the building blocks of the Pauli words used to represent qubit Hamiltonians.

Pauli term A Pauli word, sometimes considered together with its numerical coefficient in the Hamiltonian sum. The coefficient determines how strongly that Pauli word contributes to the Hamiltonian expectation value (though it is not important to determine commutativity). In our case, it is used interchangeably with Pauli word.

Pauli word A tensor product of single-qubit Pauli operators and identity operators, with one operator assigned to each qubit. Pauli words are also commonly called Pauli strings.

Penalty weight A numerical coefficient used to make constraint violations costly in a QUBO formulation.

Projective measurement The standard gate-based quantum measurement model in which a quantum state collapses to an eigenstate associated with the observed outcome.

QAOA The Quantum Approximate Optimization Algorithm, a hybrid quantum-classical algorithm for approximate combinatorial optimization.

QPU A quantum processing unit: physical quantum hardware used to run quantum circuits.

QuadraticProgram A Qiskit class for representing optimization problems with linear or quadratic objectives and constraints.

Quadratization The process of converting higher-order terms into quadratic terms, usually by adding auxiliary variables.

Qubit-wise commuting (QWC) A relation between two Pauli words where each pair of corresponding single-qubit operators either commutes directly or includes an identity. Mutually QWC Pauli words share a tensor product basis and can be grouped for measurement.

Qubit-wise commutator A function indicating whether two Pauli words are qubit-wise commuting. In this report, a value of zero indicates that the words are QWC, while a value of one indicates that they are not.

QUBO Quadratic Unconstrained Binary Optimization, a problem form where binary variables are optimized using a quadratic objective function without explicit constraints. Constraints can be included by adding penalty terms to the objective.

Qubit mapping The process of translating fermionic molecular operators into qubit operators.

Space-efficient embedding A graph colouring formulation from Tabi et al. that represents colour choices using about $\log_2(k)$ qubits per vertex rather than the k required in one-hot encoding. The space-efficient embedding reduces the number of qubits needed to represent colours, though it can introduce higher-order terms.

Tensor product basis (TPB) A measurement basis formed from the tensor product of the single-qubit bases associated with a Pauli word. Pauli words sharing the same TPB can be measured together.

Variational Quantum Eigensolver (VQE) A hybrid quantum-classical algorithm for estimating the ground-state energy of a quantum system.

References

1. Digital Research Alliance of Canada (2025), https://www.alliancecan.ca/en
2. ACENET (2026), https://ace-net.ca
3. Aaronson, S.: Shadow tomography of quantum states. SIAM Journal on Computing **49**(5), STOC18–368–STOC18–394 (2020). https://doi.org/10.1137/18M120275X, https://doi.org/10.1137/18M120275X
4. Bauckhage, C., Brito, E., Cvejoski, K., Ojeda, C., Sifa, R., Wrobel, S.: Ising models for binary clustering via adiabatic quantum computing. In: Pelillo, M., Hancock, E. (eds.) Energy Minimization Methods in Computer Vision and Pattern Recognition. pp. 3–17. Springer International Publishing, Cham (2018)
5. Benjamin-Talbot: op_min. GitHub (2026), https://github.com/Benjamin-Talbot/op_min
6. Cao, W.: Experimental verification of quantum entanglement: Insights from epr paradox and bell tests. Highlights in Science, Engineering and Technology (2023). https://doi.org/10.54097/hset.v64i.11276
7. Crawford, O., Straaten, B.v., Wang, D., Parks, T., Campbell, E., Brierley, S.: Efficient quantum measurement of Pauli operators in the presence of finite sampling error. Quantum **5**, 385 (Jan 2021). https://doi.org/10.22331/q-2021-01-20-385, https://doi.org/10.22331/q-2021-01-20-385
8. D-Wave: Advantage2 System: Our Most Advanced and Performant Quantum Computer (2026), https://www.dwavequantum.com/solutions-and-products/systems/
9. Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm. arXiv preprint arXiv:1411.4028 (2014)
10. Glos, A., Krawiec, A., Zimborás, Z.: Space-efficient binary optimization for variational quantum computing. npj Quantum Information **8**, 39 (04 2022). https://doi.org/10.1038/s41534-022-00546-y
11. Glover, F., Kochenberger, G., Du, Y.: Quantum bridge analytics i: a tutorial on formulating and using qubo models. 4or **17**(4), 335–371 (2019)
12. Gokhale, P., Angiuli, O., Ding, Y., Gui, K., Tomesh, T., Suchara, M., Martonosi, M., Chong, F.: $o(n^3)$ measurement cost for variational quantum eigensolver on molecular hamiltonians. IEEE Transactions on Quantum Engineering **1**, 1–24 (2019). https://doi.org/10.1109/tqe.2020.3035814
13. Gokhale, P., Angiuli, O., Ding, Y., Gui, K., Tomesh, T., Suchara, M., Martonosi, M., Chong, F.T.: Optimization of simultaneous measurement for variational quantum eigensolver applications. In: 2020 IEEE international conference on Quantum Computing and Engineering (QCE). pp. 379–390. IEEE (2020)
14. Hamamura, I., Imamichi, T.: Efficient evaluation of quantum observables using entangled measurements. npj Quantum Information **6**(1), 56 (2020)
15. Huggins, W.J., McClean, J.R., Rubin, N.C., Jiang, Z., Wiebe, N., Whaley, K.B., Babbush, R.: Efficient and noise resilient measurements for quantum chemistry on near-term quantum computers. npj Quantum Information **7**(1), 23 (2021)
16. IBM Quantum: Spsa optimizer — qiskit algorithms api (v0.46) (2024), https://quantum.cloud.ibm.com/docs/en/api/qiskit/0.46/qiskit.algorithms.optimizers.SPSA, accessed: 2026-04-12
17. IBM Quantum: Hardware for useful quantum computing (2026), https://www.ibm.com/quantum/hardware
18. IBM Quantum: View Backend Details - IBM Quantum Documentation (2026), https://quantum.cloud.ibm.com/docs/en/guides/qpu-information
19. Izmaylov, A.F., Yen, T.C., Lang, R.A., Verteletskyi, V.: Unitary partitioning approach to the measurement problem in the variational quantum eigensolver method. Journal of Chemical Theory and Computation **16**(1), 190–195 (2020). https://doi.org/10.1021/acs.jctc.9b00791, https://doi.org/10.1021/acs.jctc.9b00791, pMID: 31747266
20. James C. Spall: Simultaneous perturbation stochastic approximation (spsa) (nd), https://www.jhuapl.edu/SPSA/, accessed: 2026-04-12
21. Korte, B., Vygen, J.: Combinatorial optimization: theory and algorithms. Springer (2008)
22. Külske, C.: The ising model: highlights and perspectives: C. külske. Mathematical Physics, Analysis and Geometry **28**(3), 20 (2025)
23. Łuczak, T.: The chromatic number of random graphs. Combinatorica **11**(1), 45–54 (1991). https://doi.org/10.1007/BF01375472, https://doi.org/10.1007/BF01375472
24. McClean, J.R., Romero, J., Babbush, R., Aspuru-Guzik, A.: The theory of variational hybrid quantum-classical algorithms. New Journal of Physics **18**(2), 023023 (feb 2016). https://doi.org/10.1088/1367-2630/18/2/023023, https://doi.org/10.1088/1367-2630/18/2/023023
25. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge University Press (2010)
26. Perez-Ramirez, D.F.: Variational quantum algorithms for combinatorial optimization (2024), https://arxiv.org/abs/2407.06421
27. Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.H., Zhou, X.Q., Love, P.J., Aspuru-Guzik, A., O'brien, J.L.: A variational eigenvalue solver on a photonic quantum processor. Nature communications **5**(1), 4213 (2014)

28. Qiskit: Lecture 5.2 - Introduction to the Quantum Approximate Optimization Algorithm and Applications (October 2021), https://www.youtube.com/watch?v=YpLzSQPrgSc
29. QuNB-Repo: QuNB GitHub Repository. GitHub, https://github.com/QuNB-Repo
30. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Journal on Computing **26**(5), 1484–1509 (1997). https://doi.org/10.1137/S0097539795293172, https://doi.org/10.1137/S0097539795293172
31. Tabi, Z., El-Safty, K.H., Kallus, Z., Haga, P., Kozsik, T., Glos, A., Zimboras, Z.: Quantum optimization for the graph coloring problem with space-efficient embedding. In: 2020 IEEE International Conference on Quantum Computing and Engineering (QCE). pp. 56–62. IEEE (Oct 2020). https://doi.org/10.1109/qce49297.2020.00018, http://dx.doi.org/10.1109/QCE49297.2020.00018
32. Tacchino, F., Chiesa, A., Carretta, S., Gerace, D.: Quantum computers as universal quantum simulators: State-of-the-art and perspectives. Advanced Quantum Technologies **3**(3), 1900052 (2020). https://doi.org/https://doi.org/10.1002/qute.201900052, https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/qute.201900052
33. Tilly, J., Chen, H., Cao, S., Picozzi, D., Setia, K., Li, Y., Grant, E., Wossnig, L., Rungger, I., Booth, G.H., Tennyson, J.: The variational quantum eigensolver: A review of methods and best practices. Physics Reports (2021). https://doi.org/10.1016/j.physrep.2022.08.003
34. Vallury, H.J., Jones, M.A., Hill, C.D., Hollenberg, L.C.L.: Quantum computed moments correction to variational estimates. Quantum **4**, 373 (Dec 2020). https://doi.org/10.22331/q-2020-12-15-373, https://doi.org/10.22331/q-2020-12-15-373
35. Verteletskyi, V., Yen, T.C., Izmaylov, A.F.: Measurement optimization in the variational quantum eigensolver using a minimum clique cover. The Journal of chemical physics **152**(12) (2020)
36. Wecker, D., Hastings, M.B., Troyer, M.: Progress towards practical quantum variational algorithms. Phys. Rev. A **92**, 042303 (Oct 2015). https://doi.org/10.1103/PhysRevA.92.042303, https://link.aps.org/doi/10.1103/PhysRevA.92.042303
37. Yen, T.C., Izmaylov, A.F.: Cartan subalgebra approach to efficient measurements of quantum observables. PRX Quantum **2**, 040320 (Oct 2021). https://doi.org/10.1103/PRXQuantum.2.040320, https://link.aps.org/doi/10.1103/PRXQuantum.2.040320
38. Zhu, L., Liang, S., Yang, C., Li, X.: Optimizing shot assignment in variational quantum eigensolver measurement. Journal of Chemical Theory and Computation **20**(6), 2390–2403 (2024). https://doi.org/10.1021/acs.jctc.3c01113, https://doi.org/10.1021/acs.jctc.3c01113, pMID: 38483826